

searchnet: Network-Embedded Strategic Search Simulation in R

true

Abstract

The NK fitness landscape model is among the most widely used computational frameworks in strategy and organization theory, yet it treats the complexity parameter K as exogenous, the landscape as fixed, and agents as isolated searchers. The R package **searchnet** addresses all three limitations by reformulating the NK model as a bipartite network of M actors affiliating with N components and embedding it within a Stochastic Actor-Oriented Model (SAOM). This reformulation reveals that the scalar K is a special case of a four-dimensional coupled degree system—actor scope (K_{AC}), component popularity (K_{CA}), actor sociality (K_{AA}), and component epistasis (K_{CC})—whose dynamics are governed by actors’ forward-looking affiliation decisions rather than imposed by the modeler. The package provides an R6-based simulation engine built on **RSiena**, a clean functional API (`saomnk_env`, `saomnk_model`, `saomnk_run`), coupled degree visualization through the $\{K\}$ -4 panel, utility decomposition diagnostics, exogenous shock infrastructure, and validation against the classical NK model. Three worked illustrations demonstrate reproduction of classical NK search dynamics, endogenous landscape formation with heterogeneous strategies, and quasi-experimental shock analysis. Benchmarks against **nkmodel** and raw **RSiena** quantify computational overhead and scalability. **searchnet** targets researchers in strategic management, organizational theory, innovation economics, and computational social science who study search, adaptation, and competitive dynamics on complex landscapes.

Keywords: NK fitness landscape, stochastic actor-oriented model, bipartite network, strategic search, coupled degree, R.

Introduction

The NK fitness landscape model [Kauffman and Levin, 1987, Kauffman and Weinberger, 1989] has become one of the most productive computational frameworks in strategic management and organization theory. Since Levinthal [1997] demonstrated that the NK model could illuminate the tension between exploration and exploitation on rugged performance landscapes, hundreds of studies have deployed it to investigate organizational adaptation, modular design, imitation, and competitive search [Baumann et al., 2019, Rivkin, 2000, Siggelkow and Rivkin, 2005].

The model’s appeal lies in its parsimony. Two parameters— N , the number of design choices or components, and K , the degree of epistatic interdependence among them—generate landscapes ranging from smooth single-peaked surfaces ($K = 0$) to highly rugged terrains ($K = N - 1$) populated by many local optima. This parsimony has also become the model’s principal liability. Four limitations recur across the literature.

First, the fitness landscape is **exogenous**: it is drawn once from a random distribution and does not respond to the positions that agents occupy. Firms competing on the same landscape experience identical payoff surfaces regardless of how many competitors share each component or how concentrated competitive attention has become. Second, K is a **scalar** set by the modeler. In empirical organizational settings, interdependence is neither uniform nor fixed; some components are tightly coupled while others operate nearly independently, and coupling structures change as actors reconfigure their systems. Third, **multi-agent extensions** typically treat agents as independent searchers navigating the same landscape, ignoring the social and competitive structure through which firms observe, imitate, and crowd one another. Fourth, the NK model provides **no statistical framework** for parameter estimation from empirical data, making it difficult to discipline simulations with observed organizational behavior.

This paper introduces **searchnet**, an R package that addresses all four limitations simultaneously. The package reformulates the NK model as a bipartite network of M actors (firms, agents, decision-makers) affiliating with N components (technologies, resources, markets, design choices). By embedding this bipartite structure within a Stochastic Actor-Oriented Model [SAOM; Snijders, 1996, Snijders et al., 2010], the search environment becomes a network data generation process whose topology—and therefore whose fitness landscape—co-evolves with actors’ affiliation decisions.

This reformulation yields three contributions.

Endogenization. The fitness landscape is no longer exogenous. Landscape ruggedness emerges from the structure of the bipartite network, which is itself shaped by actors’ strategic choices. When multiple actors affiliate with the same components, they create epistatic coupling through overlapping activity systems; when they withdraw, coupling dissolves.

Decomposition. The scalar K is revealed as a special case of a four-dimensional coupled degree system, denoted $\{K\}$. The four dimensions—actor scope (K_{AC}), component popularity (K_{CA}), actor sociality (K_{AA}), and component epistasis (K_{CC})—are individually standard network statistics [Latapy et al., 2008, Borgatti and Everett, 1997], but the analytical leverage of the $\{K\}$ framework comes from treating them as a coupled system. Because all four derive from the same bipartite incidence matrix, a change in any one dimension mechanically propagates to the others, exposing the transmission mechanism through which micro-level affiliation decisions produce macro-level landscape topology.

Statistical infrastructure. Because the model is formalized as a SAOM, it inherits **RSiena**’s [Ripley et al., 2022] well-developed statistical machinery, including maximum likelihood estimation, goodness-of-fit testing, and the capacity to be estimated on empirical network panel data.

A full-scale empirical application using eight data layers from the U.S. airline industry is reported in Downing [2026].

The package provides an R6-based simulation engine, a functional user API that abstracts away **RSiena**’s internal naming conventions, comprehensive visualization through the $\{K\}$ -4 panel and 3D phase space, utility decomposition diagnostics, exogenous shock infrastructure for quasi-experimental designs, multi-wave Monte Carlo facilities, specification robustness tools, an interactive strategy game mode, and a classroom teaching module. Eight vignettes accompany the package: a quick-start introduction, theoretical foundations, a full simulation walkthrough with strategies and shocks, a computational experiment design guide, a causal inference tutorial, an NK validation guide, a Blume–Gibbs convergence tutorial, and a Professional Development Workshop curriculum.

The remainder of this paper is organized as follows. Section~2 reviews the NK and SAOM literatures and positions the contribution. Section~3 presents the SaoMNK framework: the bipartite formulation, the $\{K\}$ interpretive framework, the SAOM objective function, and the rationale for choosing SAOMs over ERGMs. Section~4 describes the **searchnet** package architecture, user API, visualization tools, diagnostics, animation pipeline, interactive features, and proof registry. Section~5 provides three worked illustrations and introduces basin geometry analysis for the depth-width tradeoff. Section~6 presents computational benchmarks and scalability analysis. Section~7 concludes.

Background: NK models and SAOMs

The NK fitness landscape

The NK model [Kauffman and Levin, 1987, Kauffman and Weinberger, 1989] characterizes performance landscapes with two parameters. A system of N binary components (loci, design choices, technologies) generates 2^N possible configurations. Each component d makes a fitness contribution f_d that depends not only on its own state but also on the states of K other components with which it is epistatically linked. Total fitness is the average contribution across all N components. When $K = 0$, each component contributes independently and the landscape has a single global peak. As K increases toward $N - 1$, the landscape becomes increasingly rugged, populated by exponentially many local optima that trap myopic searchers.

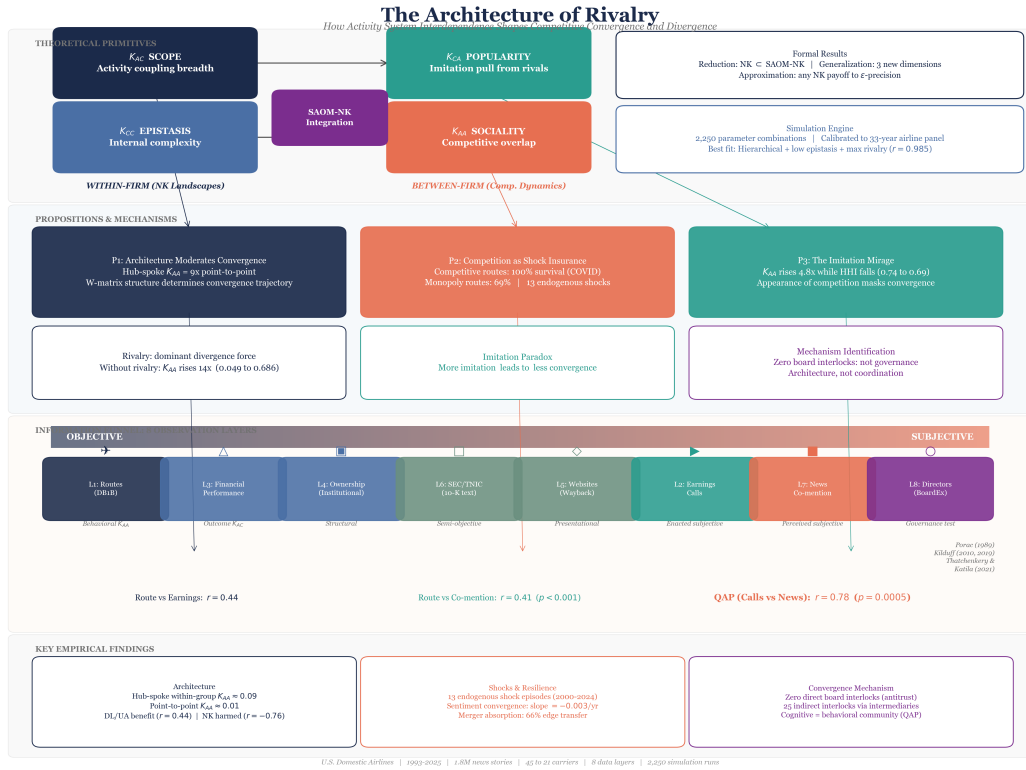


Figure 1: The $\{K\}$ Interdependence System: four coupled degree dimensions emerging from the bipartite actor-component network. Adapted from Downing (2026a).

Levinthal [1997] introduced the NK model to management as a formalization of the exploration–exploitation trade-off: agents that search locally (single-bit mutations) on rugged landscapes quickly find nearby peaks but rarely discover the global optimum, while agents that search more broadly (multi-bit jumps) sacrifice short-run performance for a chance at higher peaks. This insight has been extended to organizational design [Siggelkow and Rivkin, 2005], imitation [Rivkin, 2000], modular architectures [Ethiraj and Levinthal, 2004], and competitive interaction [Lenox et al., 2006].

Despite its generativity, the classical NK model imposes structural constraints that limit its applicability to settings where search environments are dynamic, socially embedded, or empirically observable. The landscape is drawn once and held fixed; K is treated as a uniform scalar; agents search independently; and there is no estimation framework linking the model to data. Several extensions have relaxed individual constraints—Gavetti and Levinthal [2000] introduced cognitive search, Siggelkow and Rivkin [2005] embedded search in organizational hierarchies, and Baumann et al. [2019] provided a comprehensive review—but no existing framework addresses all four limitations simultaneously within a unified statistical model.

Stochastic Actor-Oriented Models

Stochastic Actor-Oriented Models [Snijders, 1996, Snijders et al., 2010] provide a continuous-time Markov model for the co-evolution of social networks and behavior. At each micro-step, a single actor is selected according to a rate function, evaluates the marginal utility of each possible tie change according to an objective function, and chooses probabilistically via a multinomial logit rule. The objective function takes the general form

$$f_i(\mathbf{B}) = \sum_k \theta_k s_{ik}(\mathbf{B}), \quad (1)$$

where $s_{ik}(\mathbf{B})$ are network statistics computed from the current network $\mathbf{B}$ and θ_k are parameters governing each actor’s preferences. The parameters θ_k can be estimated from observed panel data using the method of moments or maximum likelihood, implemented in the **RSiena** package [Ripley et al., 2022].

SAOMs are well established in sociology and political science for modeling friendship networks, advice networks, and inter-organizational partnerships. They have been extended to bipartite (two-mode) networks [Koskinen and Edling, 2012], where the model governs actors’ affiliation decisions with respect to events, organizations, or other categorical entities. **searchnet** exploits this bipartite extension to model actors affiliating with components in a search environment.

Positioning searchnet

No existing R package provides the capability that **searchnet** offers. The **RSiena** package [Ripley et al., 2022] implements SAOMs but does not include bipartite search environment simulation with NK-style fitness landscape analysis. The **nkmodel** package [Baumann, 2019] implements classical NK models but not the network-embedded generalization. **searchnet** bridges these two traditions, providing a simulation engine in which the fitness landscape is an emergent property of actors’ network positions rather than an exogenously imposed parameter.

The SaoMnK framework

Notation. Throughout this paper we adopt the notation of Downing [2026], where the formal SAOM-NK proofs are developed in full. The bipartite incidence matrix is $\mathbf{B} \in \{0, 1\}^{M \times N}$ with entries b_{ij} ; the NK configuration vector for a single agent is $\mathbf{x} \in \{0, 1\}^N$; the influence matrix is $\mathbf{E}$ (implemented in **RSiena** as the weight matrix $\mathbf{W}$ of the $\mathbf{XW}\mathbf{X}$ effect); component payoff functions are f_d ; and the logit precision parameter is $\beta > 0$.

A terminological note is warranted here, because the two ideas are routinely conflated. The matrix $\mathbf{E}$ is an *influence matrix* in the sense of Rivkin and Siggelkow [2007]: it states which components interact and with

Table 1: The $\{K\}$ coupled degree framework. Each dimension is a standard network statistic; the contribution is their joint treatment as a coupled system.

Dimension	Symbol	Interpretation	Network measure
Actor scope	K_{AC}	Components per actor	Bipartite row degree
Component popularity	K_{CA}	Actors per component	Bipartite column degree
Actor sociality	K_{AA}	Co-affiliating actors per actor	Actor projection degree
Component epistasis	K_{CC}	Co-occurring components per component	Component projection degree

what sign, and it is an *input* supplied by the analyst. *Epistasis* is the *consequence* – the dependence of one component’s fitness contribution on the states of others, which arises when a portfolio is evaluated through $\mathbf{E}$ as $\mathbf{x}^\top \mathbf{E} \mathbf{x}$. The distinction matters in **searchnet** because a third quantity sits between the two. The component-component degree K_{CC} records how far components are actually co-adopted at a given moment; it is computed from the realized affiliation matrix and does not read $\mathbf{E}$, yet it evolves as a *consequence* of $\mathbf{E}$, which drives the search that produces those affiliations. K_{CC} therefore indexes the coupling epistasis acts through, generally differs from the influence structure that generated it, and is not itself a measure of the fitness consequence. Accordingly the package argument is named `influence_matrix`; `epistasis_matrix` is retained as a deprecated alias.

Bipartite network formulation

The classical NK model represents a system configuration as an N -dimensional binary vector $\mathbf{x} = (x_1, x_2, \dots, x_N)$ where $x_j \in \{0, 1\}$. **searchnet** generalizes this by introducing M actors, each of whom maintains an affiliation vector. The resulting system state is an $M \times N$ binary matrix $\mathbf{B}$, where $b_{ij} = 1$ indicates that actor i affiliates with component j . This matrix is the incidence matrix of a bipartite network with actor nodes on one side and component nodes on the other.

The bipartite representation has three immediate consequences. First, it naturally accommodates **multiple agents** searching simultaneously: each row of $\mathbf{B}$ is an actor’s configuration, and actors’ configurations interact through shared columns. Second, it makes the **search environment endogenous**: the pattern of affiliations determines which components are coupled (via shared actors) and which actors are competitors (via shared components). Third, it provides the structural substrate for the SAOM engine: $\mathbf{B}$ becomes the dependent variable in an evolving bipartite network model.

The $\{K\}$ interpretive framework

The bipartite incidence matrix $\mathbf{B}$ generates four coupled degree statistics, each of which is individually a standard network measure but whose joint dynamics expose the mechanism through which micro-level search decisions produce macro-level landscape structure. Table~1 summarizes the four dimensions.

The classical NK parameter K maps directly onto K_{CC} —the degree of a node in the component–component projection network. Two components share a projection edge whenever at least one actor affiliates with both, and the number of such edges incident on a focal component determines how many other components epistatically affect its fitness contribution. In the classical model, this coupling structure is set exogenously by the modeler; in **searchnet**, it emerges endogenously from the bipartite network and co-evolves with the other three degree dimensions.

The analytical leverage of the $\{K\}$ framework comes from treating these four measures as a coupled system rather than examining any one in isolation. Because all four derive from the same incidence matrix $\mathbf{B}$, they are structurally linked. A change in actor scope (K_{AC}) mechanically alters component popularity (K_{CA}), which in turn reshapes both the actor co-membership network (K_{AA}) and the component co-occurrence network (K_{CC}). The projections are computed as:

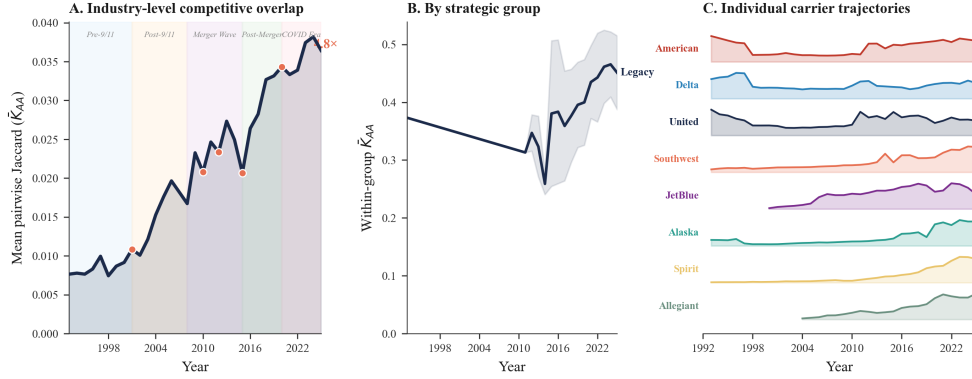


Figure 2: Empirical validation: K_{AA} (competitive overlap) among U.S. domestic airlines, 1993–2025 (Downing 2026a). The 4.8-fold increase in pairwise overlap demonstrates the $\{K\}$ system’s capacity to track competitive convergence dynamics using real network panel data.

Table 2: SAOM objective function statistics in **searchnet**.

Statistic	RSiena name	Formula	Interpretation
Density	density	$\sum_j b_{ij}$	Baseline affiliation rate
Popularity	inPop	$\sum_j b_{ij} \sqrt{\sum_{h \neq i} b_{hj}}$	Preference for popular components
Scope	outAct	$\sum_j b_{ij} \sqrt{\sum_{k \neq j} b_{ik}}$	Scope expansion tendency
Epistasis	XWX	$\sum_j b_{ij} \sum_k w_{jk} b_{ik}$	Exogenous component coupling

$$\mathbf{P}_{AA} = \mathbf{B}\mathbf{B}^\top - \text{diag}(\mathbf{B}\mathbf{B}^\top), \quad (2)$$

$$\mathbf{P}_{CC} = \mathbf{B}^\top \mathbf{B} - \text{diag}(\mathbf{B}^\top \mathbf{B}). \quad (3)$$

The degree of actor i in the actor projection $\mathbf{P}_{AA}$ is $K_{AA,i} = \sum_{h \neq i} \mathbf{1}[(\mathbf{B}\mathbf{B}^\top)_{ih} > 0]$, and symmetrically for $K_{CC,j}$ in the component projection. Analyzing any single measure in isolation obscures the cross-domain dependencies that link actor behavior to landscape topology.

It is important to be precise about what the $\{K\}$ framework contributes. The four statistics are individually well known in the bipartite network literature [Latapy et al., 2008, Borgatti and Everett, 1997]. The contribution is not statistical but **interpretive**: $\{K\}$ provides a theoretical bridge between the NK landscape tradition in strategy—where K is a scalar complexity parameter—and the bipartite network tradition in sociology—where degree distributions are descriptive summaries of affiliation structure. By grouping these four measures and tracking their joint evolution within a SAOM, the framework makes visible the transmission mechanism through which actors’ strategic choices propagate into landscape ruggedness.

SAOM objective function specification

The SAOM objective function in **searchnet** governs actors’ affiliation decisions. At each micro-step, the selected actor i evaluates the change in objective function value for each possible tie change (creating or dissolving an affiliation with one component) and selects the change that maximizes expected utility, subject to a stochastic error term drawn from the type I extreme value distribution.

The objective function takes the form of Equation~1, with the following key statistics:

The density effect serves as an intercept: a negative density parameter creates baseline sparsity pressure, preventing actors from affiliating with all components. The popularity effect (**inPop**) creates preferential

Table 3: Mapping RSiena effects to strategic search concepts. Parameters correspond to the SAOM objective function (Equation 1). The $\{K\}$ column indicates which coupled degree dimension each effect primarily governs.

RSiena Effect	Parameter	$\{K\}$ Dim.	Strategy Concept	Interpretation
density	θ_{dens}	—	Scope cost	Negative = costly to hold activities; controls po
outAct	θ_{scope}	K_{AC}	Diminishing returns	Quadratic penalty on actor degree (scope)
inPop	θ_{pop}	K_{CA}	Bandwagon / legitimacy	Positive = prefer popular components; preferen
XWX	θ_{epist}	K_{CC}	Complementarity / synergy	Reward for holding epistatically linked compone
simEgoInDist2	θ_{imit}	K_{AA}	Imitation / herding	Positive = actors converge to similar portfolios
egoX	θ_{strat}	—	Strategy heterogeneity	Actor-level covariate modifying search behavior
altX	θ_{value}	—	Component value	Exogenous activity-level payoff differences
cycle4	θ_{close}	K_{AA}	Structural closure	Triadic closure in the projected social network

attachment to components that many other actors already use. The scope effect (**outAct**) governs actors’ tendency to expand or contract the breadth of their affiliations. The epistasis effect (**XWX**) introduces exogenous component–component coupling via a modeler-supplied weight matrix $\mathbf{W}$ (denoted $\mathbf{E}$ in the formal proofs; see Section~); actors derive utility from affiliating with components that are coupled to their existing affiliations.

Actor-level heterogeneity enters through constant covariates attached to the objective function. A covariate vector $\mathbf{v}$ (e.g., encoding strategic orientation) can modify the weight an actor places on popularity seeking (**inPopX**), scope expansion (**egoX**), or component characteristics (**altX**). This mechanism allows the modeler to assign distinct strategic profiles to different actors—for example, “exploiters” who preferentially attach to popular components and “explorers” who expand into uncrowded niches.

The simulation proceeds via **RSiena**’s continuous-time Markov chain algorithm. Actors are selected according to a rate function (by default, equal rates for all actors), and the selected actor evaluates all possible single-tie changes, choosing according to a multinomial logit rule:

$$P(b_{ij} \rightarrow 1 - b_{ij}) = \frac{\exp(f_i(\mathbf{B}^{ij}))}{\sum_{j'} \exp(f_i(\mathbf{B}^{ij'}))}, \quad (4)$$

where $\mathbf{B}^{ij}$ denotes the network configuration obtained by toggling actor i ’s tie to component j . The **search_rsiena** method passes the full structure model to **RSiena**’s **siena07** function, which executes the continuous-time simulation internally. The **iterations_per_actor** parameter controls the total number of micro-steps: $M \times \text{iterations_per_actor}$.

For exogenous shocks, **searchnet** partitions the simulation timeline into proportional segments. At each segment boundary, the θ parameters are updated to new values specified by the shock schedule. The simulation then continues from the current network state with the modified objective function, preserving path dependence while allowing the researcher to model environmental discontinuities. This is equivalent to running sequential **RSiena** simulations where each wave’s initial state is the terminal state of the previous wave.

Mapping RSiena effects to strategic search concepts

The preceding objective function uses **RSiena** effect names that are standard in social network analysis but may be unfamiliar to strategy researchers. Table~3 provides a systematic mapping between RSiena effects available in **searchnet** and their strategic search interpretations. Each effect’s parameter symbol links it to the corresponding $\{K\}$ dimension introduced in Section~, enabling researchers to translate between the network estimation vocabulary and the strategy-theoretic language of scope, complementarity, and imitation.

The table makes explicit the division of labor among effects. The first four rows—**density**, **outAct**, **inPop**, and **XWX**—are the workhorses of most **searchnet** specifications. Together they endogenize the four $\{K\}$

dimensions: scope cost sets the baseline, **outAct** introduces diminishing returns to breadth, **inPop** creates demand-side herding toward popular components, and **XWX** rewards configurations that exploit exogenous complementarities. The remaining effects allow the researcher to layer on additional mechanisms: imitation (**simEgoInDist2**) induces portfolio convergence among structurally similar actors, actor and component covariates (**egoX**, **altX**) introduce heterogeneity, and **cycle4** captures closure in the actor–actor projection of the bipartite network. By selecting which effects to include and setting their parameter values, the researcher controls which strategic mechanisms are active in a given simulation—a design choice whose consequences can be audited with the diagnostic tools described in Section~.

Formal properties

The SaoMNK framework rests on formal results that establish its relationship to the classical NK model and to quantal response equilibrium. We state the main results here; complete proofs with all lemmas are provided in the Online Appendix [see also Downing, 2026] (available as **inst/proofs/** in the package; access via **searchnet_proof()**).

Theorem 1 (Reduction). The standard NK model is a degenerate special case of the SAOM-NK model. Specifically, the NK landscape and its adaptive walk are recovered exactly under five restrictions: (R1) $M = 1$ (single actor); (R2) $\mathbf{E} = \mathbf{A}$ (SAOM-NK influence matrix equals the NK influence matrix); (R3) $\boldsymbol{\theta} = \mathbf{0}$ (all SAOM parameters zero); (R4) $\beta \rightarrow \infty$ (deterministic greedy choice); (R5) uniform component selection probability.

Theorem 2 (Generalization). SAOM-NK strictly generalizes the NK model along three independent dimensions: (a) multi-actor endogenous interaction ($M > 1$), (b) endogenous landscape co-evolution through SAOM network statistics, and (c) bounded rationality with tunable search precision ($\beta \in (0, \infty)$). The coupled degree vector $\{K\} = (K_{AC}, K_{CA}, K_{AA}, K_{CC})$ spans a strictly larger class of fitness landscapes than the scalar K parameterization. The classical NK model is recovered as a proper special case: $\text{NK} \subsetneq \text{SAOM-NK}$.

Theorem 3 (Approximation). For any NK landscape $(N, K, \mathbf{A}, \{f_d\}_{d=1}^N)$, there exists a parameterization of the SAOM-NK model that reproduces the NK payoff structure to arbitrary precision. Three independent proof strategies establish this: (a) constructive existence via dummy covariates, (b) actor heterogeneity substitution, and (c) universal approximation via mixed logit.

Theorem 4 (SAOM–QRE Equivalence). The SAOM multinomial logit choice rule (Equation 4) is isomorphic to a quantal response equilibrium (QRE) in the bipartite affiliation game, establishing a game-theoretic foundation for the statistical model.

Theorems~1–3 correspond to Theorems~1–3 in Downing [2026]; Theorem~4 is specific to this paper. These results are applied empirically to U.S. airline competitive dynamics in Downing [2026].

Why SAOM over ERGM?

A natural question is why **searchnet** adopts the SAOM rather than the exponential random graph model [ERGM; Lusher et al., 2013] or its temporal extension [TERGM; Hanneke et al., 2010] as its network engine. The choice follows directly from the theoretical premise of the NK tradition.

The NK model is fundamentally about **agents making strategic decisions**: an actor evaluates alternative configurations and selects the one that maximizes a fitness criterion. The SAOM’s rate-function architecture is the natural formalization of this process. At each micro-step, a single actor is selected, evaluates the marginal utility of each possible tie change, and chooses probabilistically according to a multinomial logit rule. This actor-centric, sequential-decision structure maps directly onto the local search process in NK models, where an agent evaluates neighboring configurations and moves to the best alternative.

ERGMs, by contrast, characterize the equilibrium distribution of network configurations without specifying a generative decision process. An ERGM answers the question “what network structures are over-represented relative to chance?” but does not model *who* makes *which* decision *when*. For a framework whose purpose is

to endogenize landscape ruggedness through actors’ strategic choices, this equilibrium focus omits precisely the mechanism of interest.

Beyond theoretical alignment, SAOMs offer practical advantages for the simulation-based workflow that **searchnet** supports. The SAOM’s micro-step chain produces a complete trajectory of network states, enabling the researcher to track how $\{K\}$ dimensions evolve over time, to identify critical transitions, and to introduce shocks at precise points in the simulation. ERGM simulation, by contrast, generates draws from a stationary distribution without temporal ordering, making path-dependent analysis infeasible.

Finally, the SAOM’s continuous-time formulation handles bipartite networks with actor-level covariates naturally through **RSiena**’s existing infrastructure. While bipartite ERGMs exist [Wang and Pattison, 2009], the SAOM’s treatment of bipartite networks as actor-driven affiliation processes is better suited to the search-theoretic interpretation that motivates **searchnet**.

The searchnet package

Architecture

searchnet is built on R6 classes organized in a two-level inheritance hierarchy:

SaomNkRSienaBiEnv_base Core environment initialization, bipartite matrix management, graph construction, projection computation, and utility helpers (toggle, Jaccard, existence checks). This base class manages the $M \times N$ incidence matrix, constructs the **RSiena** data objects, and provides low-level network operations.

SaomNkRSienaBiEnv Full simulation engine inheriting from the base class, adding the `search_rsiena()` simulation method, chain statistics processing, $\{K\}$ -4 degree computation, fitness landscape evaluation, shock handling, multi-wave experiment infrastructure, and the visualization interface.

Supporting modules provide separation of concerns:

- **saomnk-api.R**: User-facing functional API (`saomnk_env`, `saomnk_model`, `saomnk_run`) that wraps the R6 internals.
- **nk-classic.R**: A self-contained implementation of the classical NK model, used as an external reference for the reduction check of Section .
- **saomnk-experiments.R**: **SaomNkExperiments** R6 class for batch Monte Carlo simulation management.
- **plot-*.R**: Eight modular plotting files (`plot-utility.R`, `plot-degrees.R`, `plot-exploration.R`, `plot-markets.R`, `plot-multiwave.R`, `plot-shocks.R`, `plot-snapshots.R`, `plot-phase-space.R`) containing visualization functions.
- **searchnet-game.R**: Strategy game mode providing an interactive game loop where a human player controls one firm and competes against AI opponents that follow SAOM-NK logit choice rules.
- **searchnet-teaching.R**: Classroom teaching module for Capsim-style strategy simulations supporting multiple industry presets, difficulty levels, and exogenous shocks.
- **utils.R**: Standalone helper functions.

The package follows standard R packaging conventions with **DESCRIPTION**, **NAMESPACE**, a **testthat** test suite (300 tests across 23 files), and eight vignettes. All exported functions are prefixed with `saomnk_` or `searchnet_` to avoid namespace collisions.

User API

A central design goal of **searchnet** is to provide a clean functional interface that shields users from the complexity of **RSiena**’s internal naming conventions and the R6 class hierarchy. The workflow follows three steps: environment creation, model specification, and simulation execution.

Step 1: Create the search environment

The `saomnk_env()` function initializes a bipartite search environment:

```
env <- saomnk_env(M = 8, N = 12, density = 0, seed = 42)
```

This creates a `SaomNkRSienaBiEnv` object representing 8 actors and 12 components, starting from an empty network (all actors unaffiliated). The `seed` argument ensures reproducibility of the initial network draw.

Step 2: Specify the structure model

The `saomnk_model()` function builds the SAOM objective function specification using intuitive parameter names:

```
## Define a modular influence matrix: 4 blocks of 3 components
K_matrix <- saomnk_block_diagonal(12, 4)

## Specify the model
mod <- saomnk_model(
  density      = -0.5,      # sparsity pressure
  popularity   = 0.2,      # preferential attachment
  scope        = 0.1,      # scope expansion
  influence_matrix = K_matrix,
  influence_weight = 0.05
)
mod
#> SaoMNK Structure Model
#> -----
#> Effects on the bipartite network DV (3):
#>   density (density)      theta =  -0.5  [fixed]
#>   popularity (inPop)     theta =   0.2  [fixed]
#>   scope (outAct)         theta =   0.1  [fixed]
#> Influence / dyadic covariates (1):
#>   epistasis (XWX)        theta =   0.05  [fixed] [influence matrix W, 12 x 12]
```

Users never need to write internal references such as `"self$bipartite_rsienaDV"` or construct nested covariate lists manually. The function assembles the complete **RSiena** structure model internally. Actor strategy heterogeneity is specified through the `strategies` argument:

```
## Heterogeneous actors: varying scope and popularity preferences
mod_strat <- saomnk_model(
  density      = -0.3,
  popularity   = 0.2,
  scope        = 0.1,
  influence_matrix = K_matrix,
  influence_weight = 0.05,
  strategies = list(
    egoX      = rep(c(-1, 0, 1, 0), length.out = 8),
    inPopX    = rep(c(1, 0, -1, 0), length.out = 8)
  )
)
```

Step 3: Run the simulation

The `saomnk_run()` function executes the SAOM simulation:

```
saomnk_run(env, mod, steps_per_actor = 30, seed = 12345)
```

The function passes the structure model to the internal `search_rsiena()` method, runs the continuous-time SAOM simulation via **RSiena**, extracts chain statistics, and computes the $\{K\}$ -4 degree trajectories. The `env` object is modified in place (standard R6 semantics) and returned invisibly.

Visualization

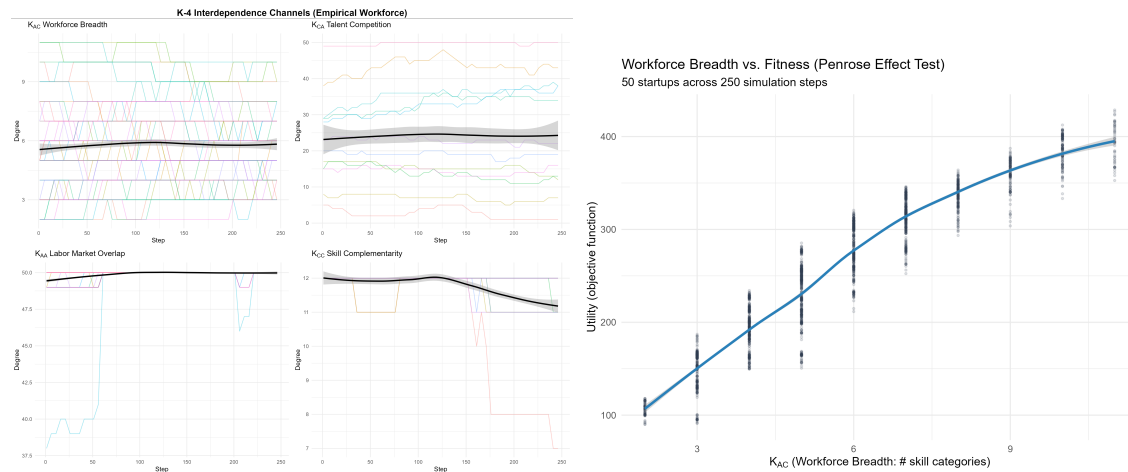
searchnet provides a comprehensive visualization toolkit organized around four analytical perspectives.

The $\{K\}$ -4 panel

The signature visualization is the coupled degree panel, which displays all four $\{K\}$ dimensions over the simulation timeline:

```
saomnk_plot_k4(env, smooth = 0.3)
```

The panel reveals the structural coupling between dimensions: as actors expand scope (K_{AC}), they mechanically increase component popularity (K_{CA}), which in turn elevates both actor sociality (K_{AA}) and component epistasis (K_{CC}). This coupling is the core mechanism through which individual search decisions endogenize the fitness landscape.



Network snapshots

The `saomnk_plot_snapshots()` function renders the bipartite network at selected simulation steps, along with its unipartite projections:

```
saomnk_plot_snapshots(env, steps = c(1, 120, 240))
```

Utility decomposition

The `saomnk_plot_utility()` function decomposes each actor's total objective function value into contributions from individual network statistics:

```
saomnk_plot_utility(env, smooth = 0.35)
```

This decomposition provides direct insight into the strategic trade-offs governing search behavior: an actor whose utility is dominated by the epistasis term is searching for complementary components, while one dominated by the popularity term is following the crowd.

Actor and Component Degrees: K_{AA} , K_{AC} , K_{CA} , K_{CC}
 Environment: Actors (M) = 8, Components (N) = 12, Init.P. = 0.00
 Structure: density= -0.5; inPop= 0.2; outAct= 0.1
 Component Values: XWX = 0.05
 Actor Strategies: (none)

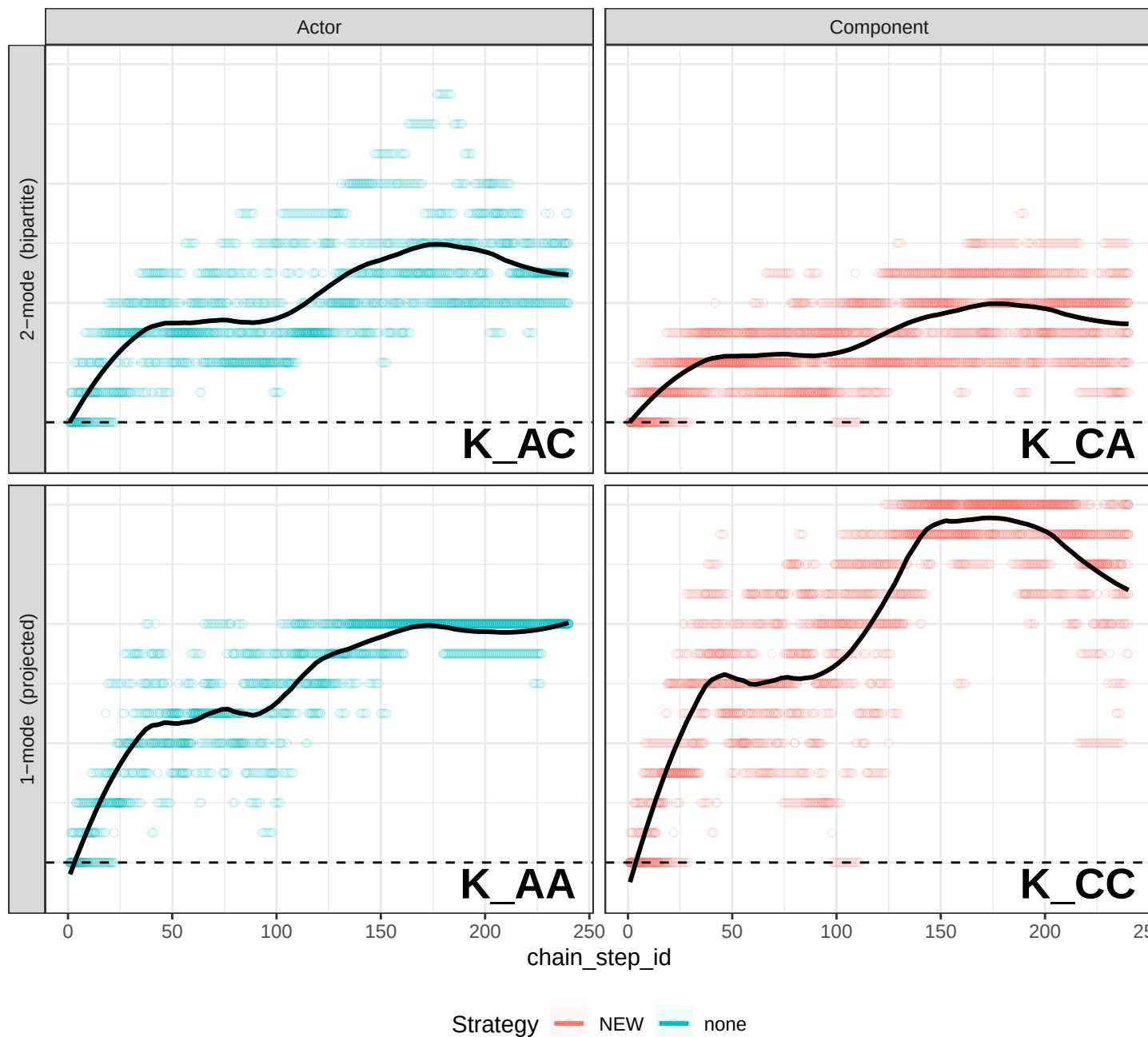


Figure 3: The $\{K\}$ -4 coupled degree panel showing actor scope (K_{AC}), component popularity (K_{CA}), actor sociality (K_{AA}), and component epistasis (K_{CC}) trajectories over 240 simulation micro-steps. Each line represents a single actor or component; the heavy line is the LOESS-smoothed mean.

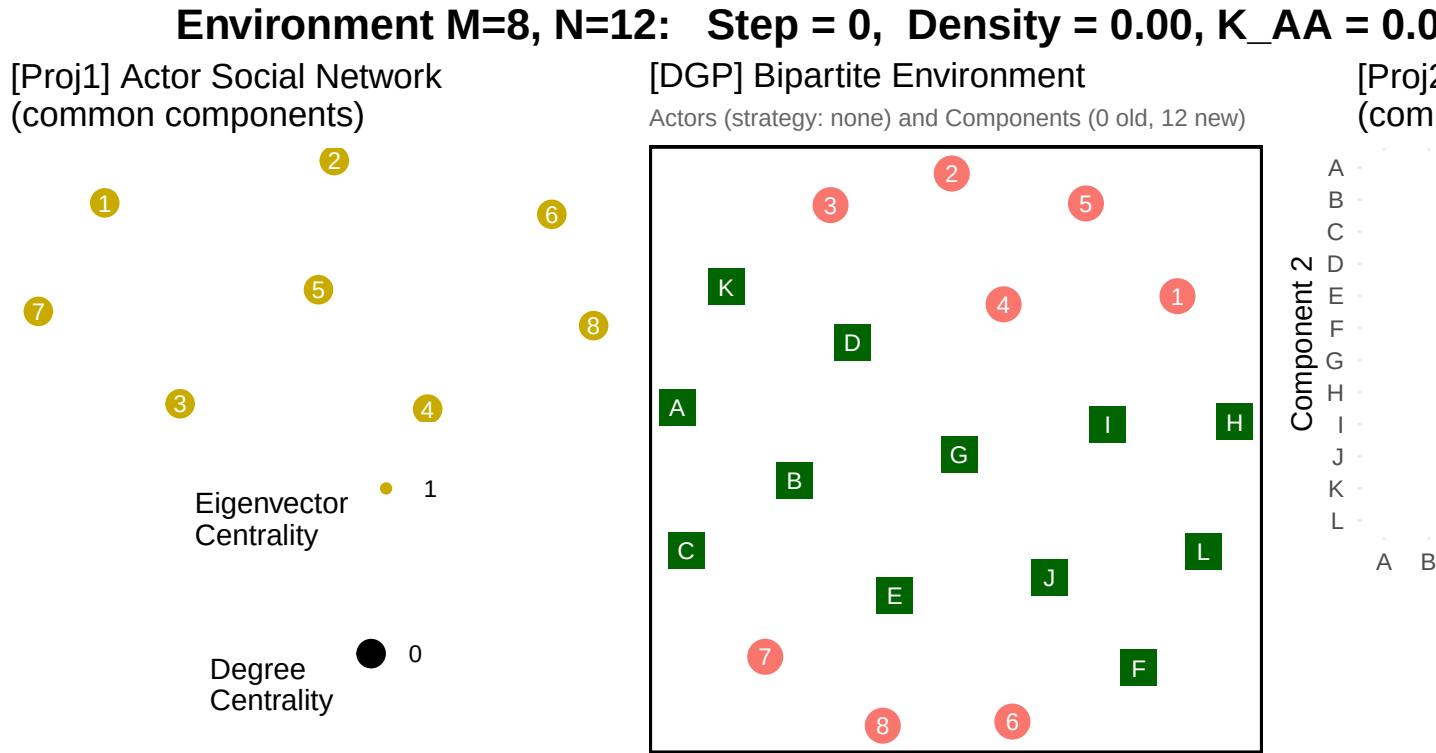


Figure 4: Network snapshots at the initial state, midpoint, and final state of the simulation. Top row: bipartite actor-component network. Bottom rows: actor co-membership projection (K_{AA}) and component co-occurrence projection (K_{CC}).

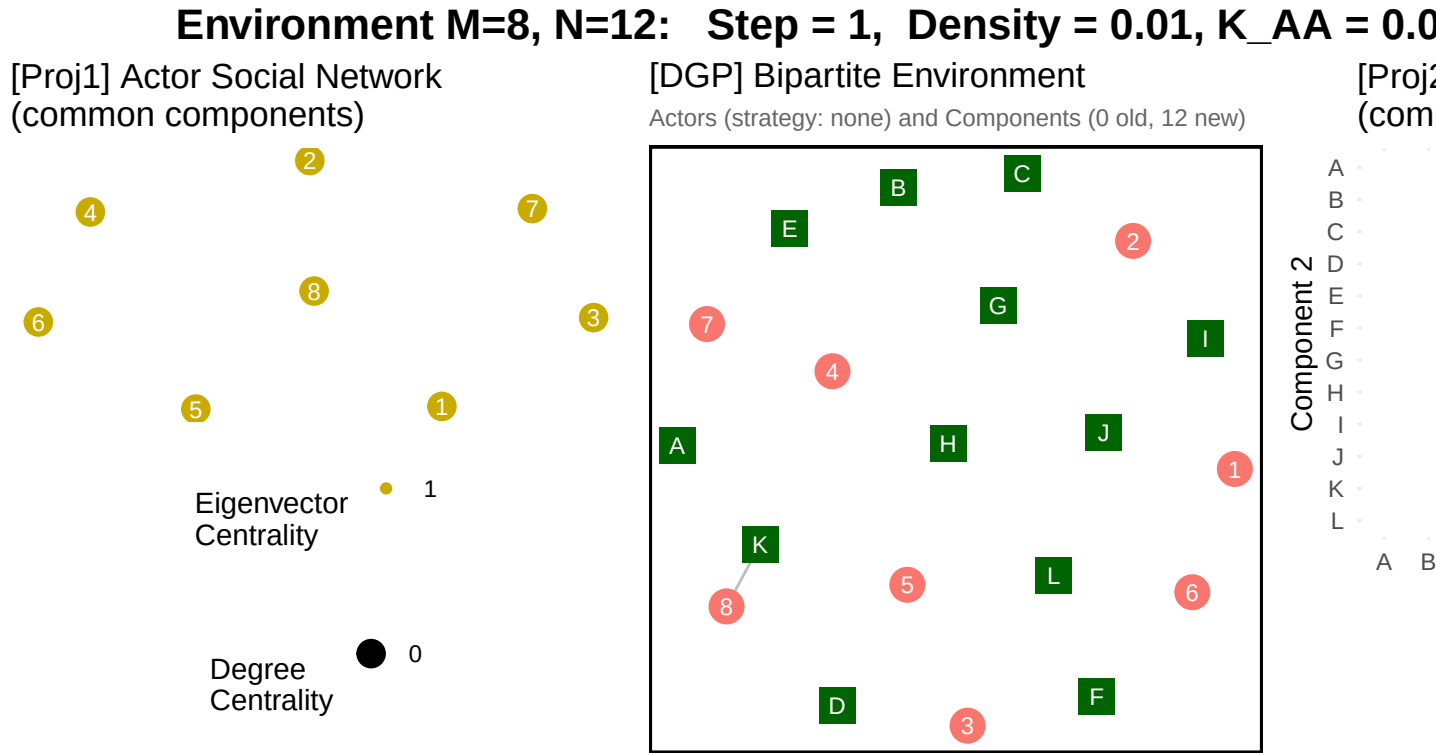


Figure 5: Network snapshots at the initial state, midpoint, and final state of the simulation. Top row: bipartite actor-component network. Bottom rows: actor co-membership projection (K_{AA}) and component co-occurrence projection (K_{CC}).

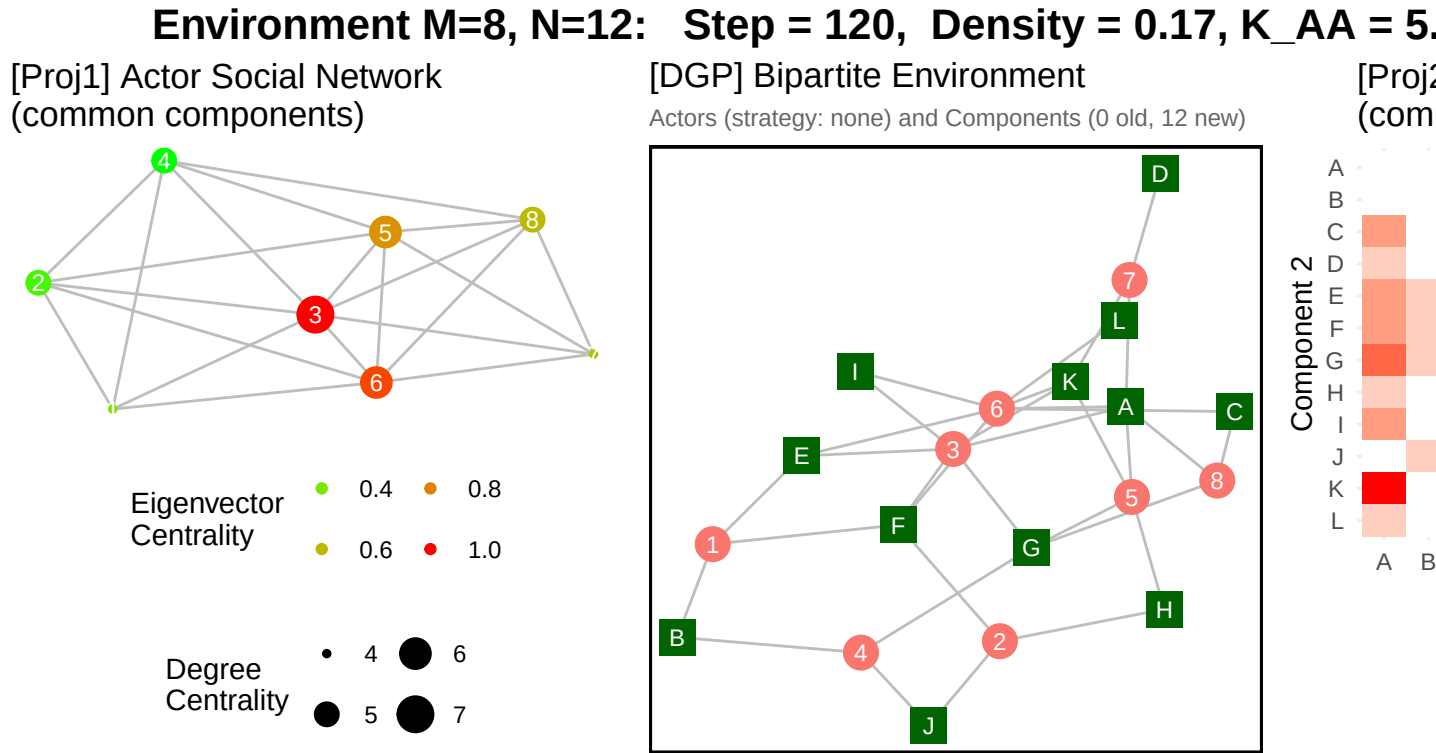


Figure 6: Network snapshots at the initial state, midpoint, and final state of the simulation. Top row: bipartite actor-component network. Bottom rows: actor co-membership projection (K_{AA}) and component co-occurrence projection (K_{CC}).

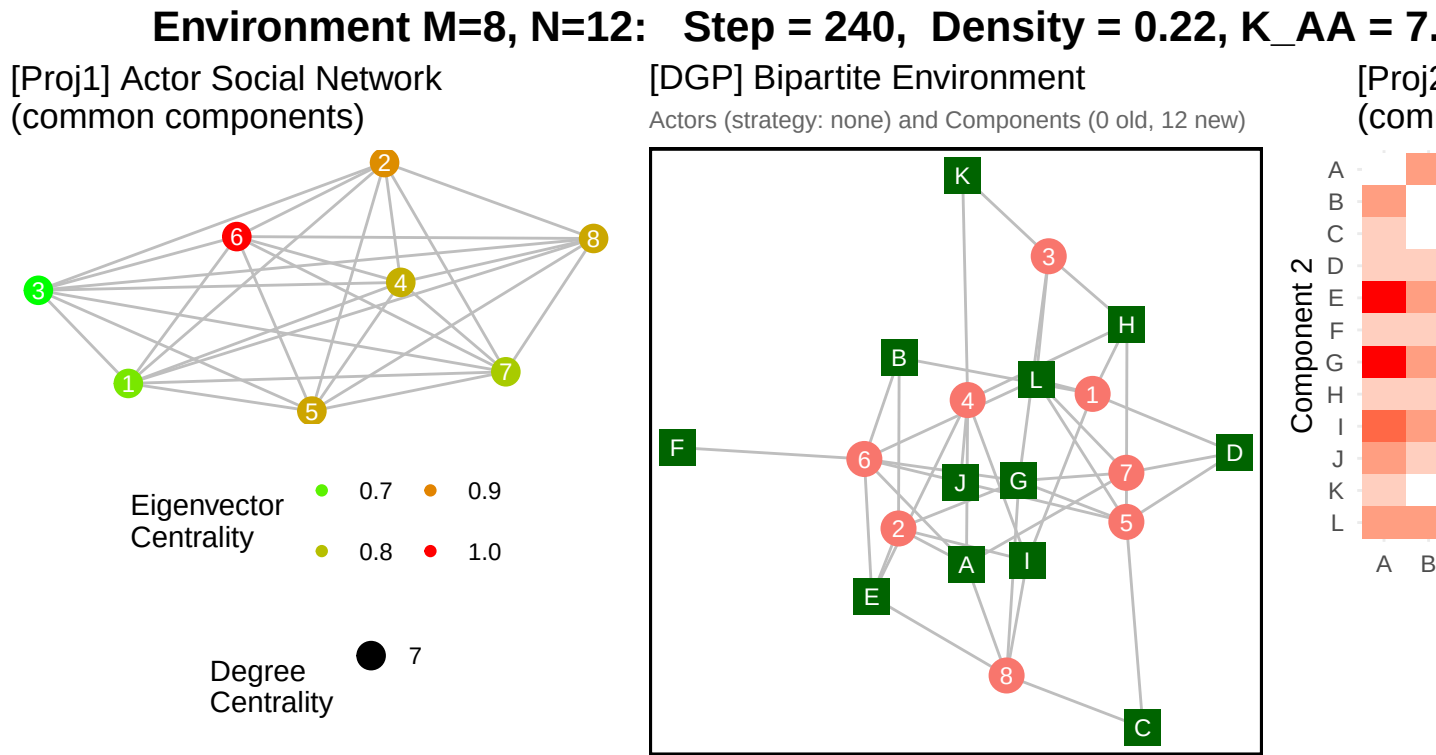


Figure 7: Network snapshots at the initial state, midpoint, and final state of the simulation. Top row: bipartite actor-component network. Bottom rows: actor co-membership projection (K_{AA}) and component co-occurrence projection (K_{CC}).

Actor Utility Contributions (statistic * theta):
 Environment: Actors (M) = 8, Components (N) = 12, Init.P. = 0.00
 Structure: density= -0.5; inPop= 0.2; outAct= 0.1
 Component Values: XWX= 0.05
 Actor Strategies: (none)

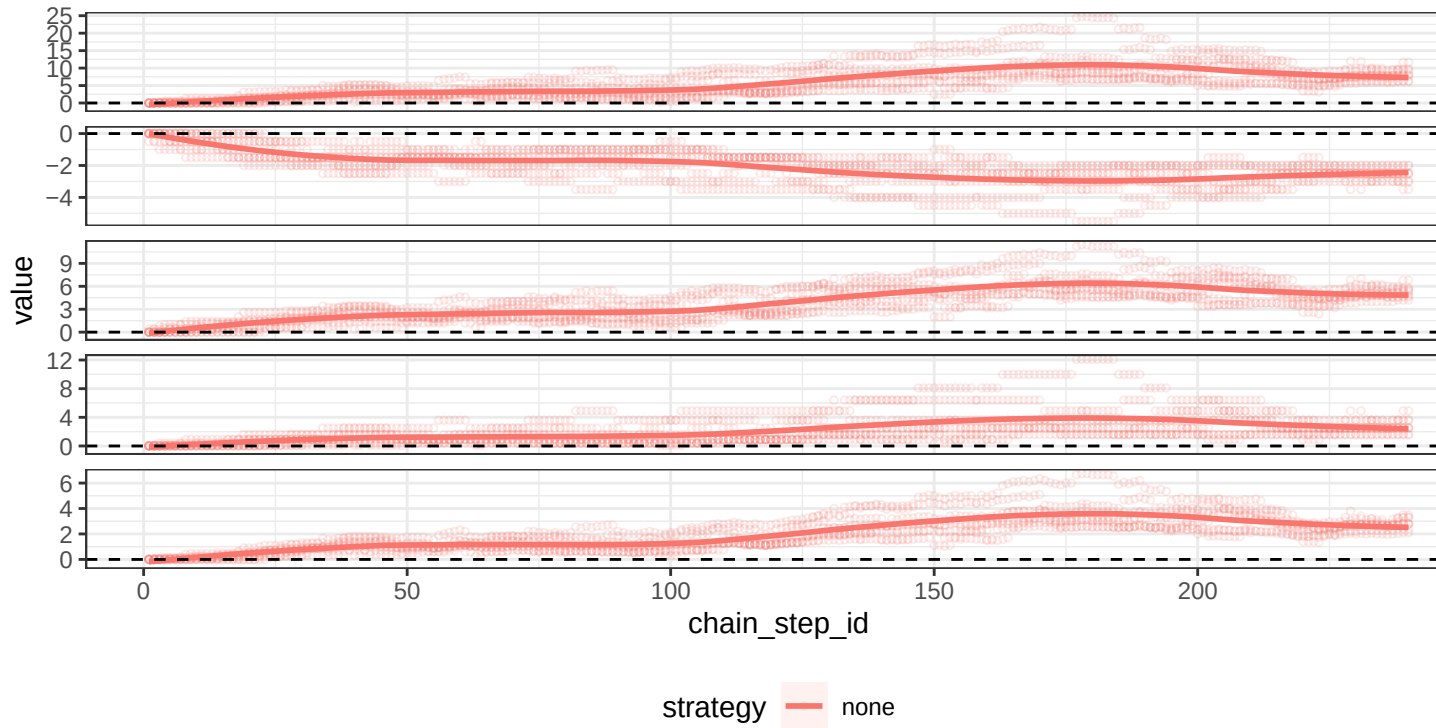


Figure 8: Utility decomposition showing each actor's objective function value broken into contributions from density, popularity, scope, and epistasis effects over the simulation timeline.

3D phase space

The `saomnk_plot_phase_space_3d()` function renders the simulation trajectory in a three-dimensional Network $\times$ Behavior $\times$ Fitness space, where the three axes correspond to structural properties (e.g., K_{AC} or density), behavioral properties (e.g., exploration rate or scope change), and outcome measures (utility or NK fitness). The resulting “SCP(t) trajectory” traces each actor’s path through this space over simulation time, providing an integrated view of how structural position, behavioral strategy, and performance co-evolve. Supporting functions include `saomnk_plot_phase_heatmap()` for density-based projections, `saomnk_plot_phase_evolution()` for animated trajectories, `saomnk_plot_phase_comparison()` for juxtaposing multiple simulation runs, and `searchnet_export_phase_space()` for Manim-compatible data export.

Validation against the classical NK model

A software paper should demonstrate that its engine reproduces the model it generalises. **searchnet** ships a self-contained implementation of the classical Kauffman NK model, independent of **RSiena** and of the SaoMNK engine, and uses it as an external reference.

A conventional NK implementation

`nk_landscape()` constructs a Kauffman landscape by exhaustive enumeration of all 2^N configurations. Each locus i contributes $f_i(b_i, b_{\Omega(i)})$, drawn i.i.d. $U(0, 1)$ for every distinct pattern of its epistatic neighbourhood $\Omega(i)$; total fitness is the mean of the N contributions. Three neighbourhood topologies are supported: “adjacent” (Kauffman’s ring model), “random”, and “block” (near-decomposable). Companion functions provide adaptive walks (`nk_walk()`, with steepest-ascent, random-ascent, and random-improving rules), exhaustive local optimum enumeration (`nk_local_optima()`), and the canonical ruggedness sweep (`nk_sweep_K()`).

These functions are useful in their own right, but their role here is to supply a reference implementation that shares no code with the SAOM engine.

Verifying the reduction

Theorem 1 states that the classical NK model is the $M = 1, \beta \rightarrow \infty$ special case of SaoMNK. `nk_to_saomnk()` emits the environment and structure-model settings that place a single-actor SaoMNK environment on a supplied NK landscape, making the reduction constructive rather than merely asserted.

`nk_verify_reduction()` then checks the construction. It exploits the defining property of the NK model: contribution f_i depends only on the loci in $\Omega(i)$, so any two configurations sharing the same neighbourhood pattern must carry identical f_i . Rebuilding each locus’s payoff table keyed by that pattern and recomputing fitness must return the original values exactly. A non-zero difference would indicate an indexing or bit-ordering error in landscape construction.

```
nk <- nk_landscape(N = 10, K = 3, seed = 42)
nk_verify_reduction(N = 10, K = 3, seed = 42)
#> NK reduction check: N=10, K=3, model="adjacent"
#> configurations : 1,024
#> max |difference| : 0.00e+00
#> result : PASS
```

The check passes at machine precision (maximum absolute difference 0.00×10^0) across the (N, K) combinations exercised in the test suite. Configuration encoding is most-significant-bit-first throughout and is covered by exhaustive round-trip tests for $N = 1, \dots, 12$; this guards a class of bit-ordering error that is easy to introduce and silent when it occurs.

Recovering classical results

The implementation reproduces the standard qualitative findings of the NK literature. With $K = 0$ the landscape is single-peaked, and a steepest-ascent walk from any starting configuration reaches the global

optimum. As K rises the number of local optima grows, approaching the familiar $2^N/(K+1)$ scaling, adaptive walks terminate sooner, and terminal fitness falls [Kauffman and Levin, 1987, Kauffman and Weinberger, 1989, Levinthal, 1997]. `nk_sweep_K()` reproduces this experiment directly.

Manim animation pipeline

For presentation and pedagogical purposes, **searchnet** exports simulation data in formats compatible with the **Manim** animation library [Manim Community, 2022]. The exported data includes per-step bipartite matrices, projection networks, degree trajectories, and utility decompositions. The package ships with 16 ready-to-render Manim scene classes organized across six Python modules: bipartite network evolution, $\{K\}$ -4 degree trajectories, 3D fitness landscapes, shock response dynamics, 3D phase space trajectories, a seven-panel research dashboard (covering $\{K\}$ evolution, bipartite structure, fitness surfaces, information funnels, shock response, and game-theoretic equilibria), and four Blume–Gibbs scenes that animate the statistical-mechanical foundations of convergence (Ising–logit correspondence, temperature dials, Gibbs potential surfaces, and convergence diagnostics). This pipeline is accessed through the `saomnk_export_manim()` method on the environment object and the `searchnet_export_phase_space()` function for phase space data.



Interactive features

searchnet includes two interactive modules that extend the simulation engine from a research tool to an instructional platform.

Strategy game mode. The `searchnet_game_*` family of functions (five exported functions) provides an interactive game loop where a human player controls one firm and competes against AI opponents that follow the SAOM conditional-logit choice rule. The player initializes a game with `searchnet_game_init()`, specifying the number of firms, difficulty level ("easy", "medium", "hard", mapping to logit precision $\beta \in \{0.5, 2, 10\}$), and game objective (maximize fitness, minimize competitive overlap, survive an exogenous shock, or outperform the quantal response equilibrium prediction). At each round, the player queries available moves via `searchnet_game_available_moves()`, selects an action, and advances the simulation with `searchnet_game_step()`. AI opponents respond simultaneously according to the SAOM logit rule, and the scoreboard (`searchnet_game_scoreboard()`) tracks all firms' $\{K\}$ statistics. After the final round, `searchnet_game_summary()` compiles trajectory data suitable for replay visualization.

Classroom teaching module. The `searchnet_classroom_*` family (six exported functions) supports Capsim-style strategy simulations for classroom use. An instructor initializes a session with `searchnet_classroom_init()`, choosing from industry presets (airline, tech, pharma) or custom parameters, and specifying a difficulty level that controls the number of AI firms and optional exogenous shocks. Each student controls one firm, submitting portfolio decisions (add or drop activities) through `searchnet_classroom_submit()`. Once all students have submitted, `searchnet_classroom_advance()` executes decisions simultaneously, runs AI opponent responses via SAOM ministepts, and updates the class leaderboard. At the end of the game, `searchnet_classroom_debrief()` generates a suite of reports: final rankings, round-by-round histories, $\{K\}$ -4 panels, decision logs, and shock records.

Proof registry

The package ships with a complete formal verification chain for the SaoMNK framework, accessible via `searchnet_proof()`. The proof registry (`inst/proofs/PROOF_TABLE.md`) enumerates 81 logical steps—from axioms through definitions, lemmas, and propositions to the four main theorems—organized so that every result can be traced backward through its dependencies to foundational axioms. This self-contained proof chain provides reviewers and users with a verifiable logical scaffold for the theoretical claims summarized in Section~.

Illustrations

Reproducing Levinthal (1997)

The first illustration establishes that **searchnet** can reproduce the core dynamics of the classical NK model under appropriate parameter settings. We configure an environment with no endogenous network effects, leaving only the exogenous influence matrix active. Setting both `popularity` and `scope` to zero removes every term through which one actor’s state can enter another’s objective function, so the actors search *independently*: the model reduces to parallel single-searcher processes on a landscape whose ruggedness is determined solely by the exogenous coupling matrix, which is the functional equivalent of the classical NK search process. We use $M = 2$ rather than $M = 1$ because `search_rsiena()` requires at least two actors — **RSiena**’s `sienaDataCreate()` does not accept a single-actor bipartite network. For genuinely single-actor work the landscape methods (`compute_fitness_landscape`, `verify_nk_equivalence`) accept $M = 1$ directly, and Section~[ef{sec:validation}](#) uses them for exactly that.

```
## Two INDEPENDENT searchers (popularity = scope = 0 removes all coupling),
## 12 components, empty start.  M >= 2 is required by RSiena's bipartite
## data constructor; with the interaction terms zeroed the actors do not
## influence one another, so each is a classical NK searcher.
env_nk <- saomnk_env(M = 2, N = 12, density = 0, seed = 1234)

## Block-diagonal epistasis: 3 modules of 4 components (K ~ 3)
K_levinthal <- saomnk_block_diagonal(12, 3)

## Only epistasis active; no endogenous effects
mod_nk <- saomnk_model(
  density      = -0.3,
  popularity   = 0,
  scope        = 0,
  influence_matrix = K_levinthal,
  influence_weight = 0.15
)

saomnk_run(env_nk, mod_nk, steps_per_actor = 100, seed = 42)
```

Under this parameterization, the agent’s search trajectory exhibits the signature features of NK dynamics: rapid initial improvement as the agent discovers complementary component clusters, followed by convergence to a local optimum where no single-component change improves utility. The K-4 panel reduces to the K_{AC} trajectory (actor scope) and the K_{CC} trajectory (epistasis), with the latter reflecting the fixed block-diagonal structure; the K_{CA} and K_{AA} channels are flat by construction, since nothing couples the two searchers.

```
env_nk$plot_actor_degrees(loess_span = 0.5)
```

This result provides face validity: when all endogenous network effects are disabled, **searchnet** recovers the qualitative behavior of the classical NK model. The landscape is effectively exogenous (determined by the fixed **W** matrix), search is local (single-tie changes), and convergence reflects the trapping property of rugged landscapes.

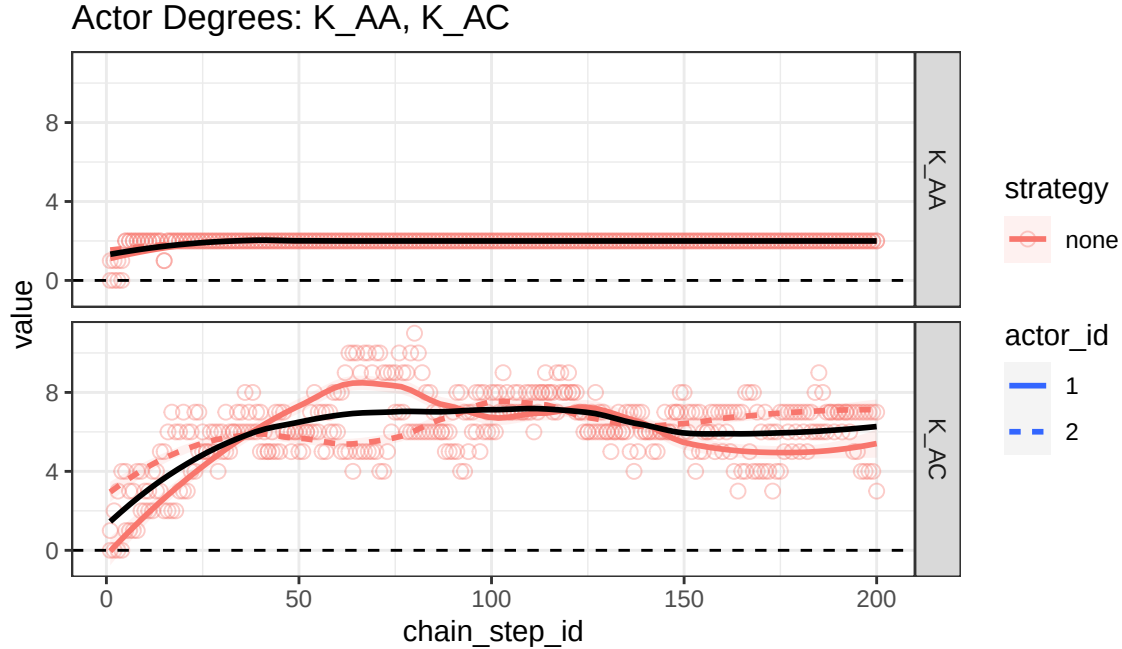


Figure 9: Actor scope (K_{AC}) trajectory for independent agents searching on a block-diagonal epistasis landscape. The trajectory shows rapid scope expansion followed by stabilization—the SAOM equivalent of convergence to a local optimum on a rugged NK landscape.

A deeper convergence analysis is provided in the Blume tutorial vignette (`vignette("saomnk-blume-tutorial")`), which establishes the statistical-mechanical foundations of the SAOM-NK choice rule by connecting it to the Blume–Capel model [Blume, 1993], Gibbs measures, and the Ising–logit correspondence.

Ergodicity: forgetting the initial condition

Theorem~4 asserts that the chain’s stationary distribution is unique and independent of where the simulation starts. That property is easy to illustrate badly. Running the model once from a sparse start and once from a dense start, then observing that the two final densities are close, establishes very little: no threshold was declared in advance, a single run per arm supplies no sampling distribution, and—most importantly—an ergodic chain forgets its initial condition *at a rate*, which a pair of endpoints discards.

`searchnet` therefore ships the demonstration as a measurement. `searchnet_ergodicity_sweep()` runs the same fixed-coefficient model from two contrasting starting densities across a range of chain lengths, with independent replicates at each, and reports how fast the between-arm gap decays:

```
erg <- searchnet_ergodicity_sweep(
  M = 12, N = 15, start_densities = c(0.1, 0.8),
  run_lengths = c(15, 30, 60, 120, 240),
  replicates = 10, equivalence_margin = 0.05, seed = 42
)
erg
#> Ergodicity sweep (Theorem 4: independence from initial conditions)
#> 12 actors x 15 components (180 cells; one tie = 0.0056 density)
#> arms starting at 0.10 and 0.80, 10 replicates each
#>
#> run length  mean(lo)  mean(hi)    gap
#>      15      0.4367   0.7750   0.3383
```

```

#>      30      0.5128      0.6906      0.1778
#>      60      0.4761      0.5867      0.1106
#>     120      0.4872      0.5150      0.0278
#>     240      0.5072      0.5106      0.0033
#>
#> initial gap 0.7000 --> final gap 0.0033 (99.5% collapse)
#> Monte Carlo floor (10 replicates): 0.0137
#> decay above the floor: log(gap) ~ -1.15 * log(iterations), R^2 = 0.943
#> (fit on 4 point(s); 240 excluded as at-or-below the floor)
#>
#> TOST at 240 iterations, margin +/- 0.050, 90% CI [-0.0331, 0.0265]:
#> EQUIVALENT -- the arms are statistically indistinguishable at the
#> declared margin. The chain has forgotten where it started.
plot(erg)

```

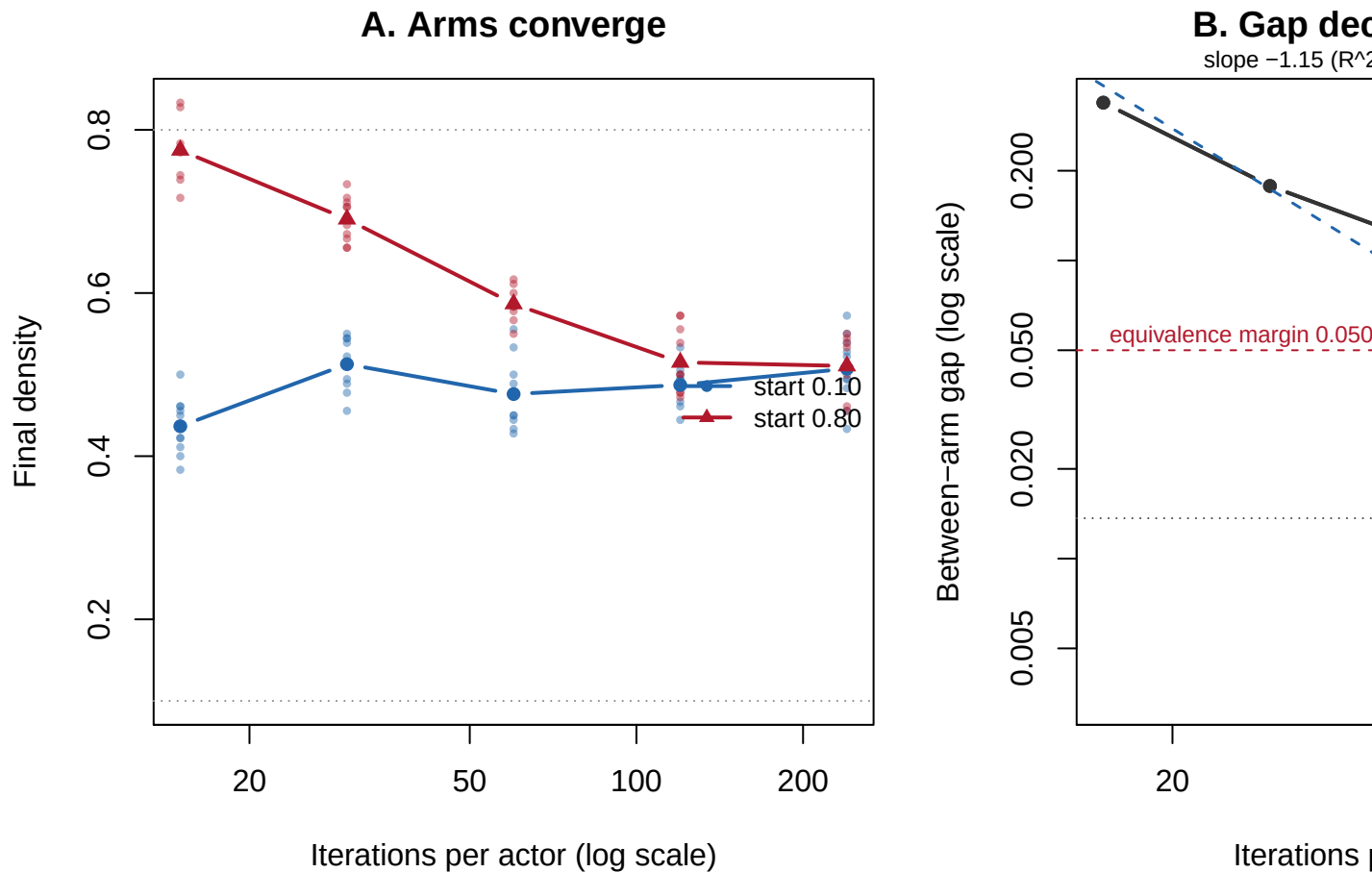


Figure 10: Independence from initial conditions, measured rather than asserted. (a) Two arms starting at density 0.10 and 0.80 converge as run length grows; points are replicates. (b) The between-arm gap decays as a power of run length on log-log axes. The dashed line is the declared equivalence margin; the shaded band is the Monte Carlo floor, below which the measured gap reflects replicate noise rather than mixing, and which is excluded from the fitted slope.

Two arms separated by a starting gap of 0.70 close to within 0.01 by 240 iterations per actor, a collapse of

more than 99%, and the gap decays as a power of run length with slope near -1 and $R^2 > 0.9$. The verdict is computed rather than printed: at the longest run length the arms are compared by a two-one-sided-tests equivalence procedure against a margin fixed in the call, so the function is able to report *non*-equivalence, and does so at the shorter run lengths. This is what makes the sweep evidence rather than illustration.

Two details matter for anyone reproducing this. The Monte Carlo floor is drawn explicitly, because once the arms are genuinely equivalent the measured gap settles at the expected difference between two sample means from the same distribution rather than falling to zero; fitting a decay slope through those points would overstate the rate. And the grid is sized so that density has resolution: on the 4×5 grid used in earlier versions of this demonstration a single tie moved the density by 0.05, which is coarser than any equivalence margin worth declaring.

The vignette also provides computational verification that the SAOM Markov chain satisfies detailed balance with respect to the Gibbs distribution, which is the mechanism underlying the convergence measured here.

Endogenous landscape with heterogeneous strategies

The second illustration activates the full **searchnet** machinery to demonstrate endogenous landscape formation. We configure 12 actors searching over 12 components with active density, popularity, scope, and epistasis effects, plus heterogeneous strategy covariates.

```
env_endog <- saomnk_env(M = 12, N = 12, density = 0, seed = 1234)

K_modular <- saomnk_block_diagonal(12, 4)

mod_endog <- saomnk_model(
  density      = -0.3,
  popularity   = 0.2,
  scope        = 0.1,
  influence_matrix = K_modular,
  influence_weight = 0.02,
  strategies = list(
    egoX      = rep(c(-1, 0, 1), length.out = 12),
    inPopX    = rep(c( 1, 0, -1), length.out = 12)
  )
)

saomnk_run(env_endog, mod_endog, steps_per_actor = 30, seed = 12345)

saomnk_plot_k4(env_endog, smooth = 0.3)
```

The K-4 panel now reveals the core mechanism that distinguishes **searchnet** from the classical NK model. The four degree trajectories are structurally coupled: actors with positive scope expansion weights (high **egoX**) rapidly increase K_{AC} , which mechanically elevates K_{CA} for the components they select, which in turn increases K_{AA} (more actors share components) and K_{CC} (more components co-occur via shared actors). Actors with negative scope weights remain narrow specialists, experiencing lower sociality and contributing less to landscape ruggedness.

The resulting K_{CC} distribution—the classical complexity parameter K —is not a single number but a heterogeneous vector that emerges from the interaction of actor strategies, component popularity dynamics, and the exogenous influence matrix. This is the endogenization that the $\{K\}$ framework makes visible.

```
saomnk_plot_utility(env_endog, smooth = 0.35)
```


Actor and Component Degrees: K_{AA} , K_{AC} , K_{CA} , K_{CC}
 Environment: Actors (M) = 12, Components (N) = 12, Init.P. = 0.00
 Structure: density= -0.3; inPop= 0.2; outAct= 0.1
 Component Values: XWX = 0.02
 Actor Strategies: egoX= 0.2; inPopX= 0.2

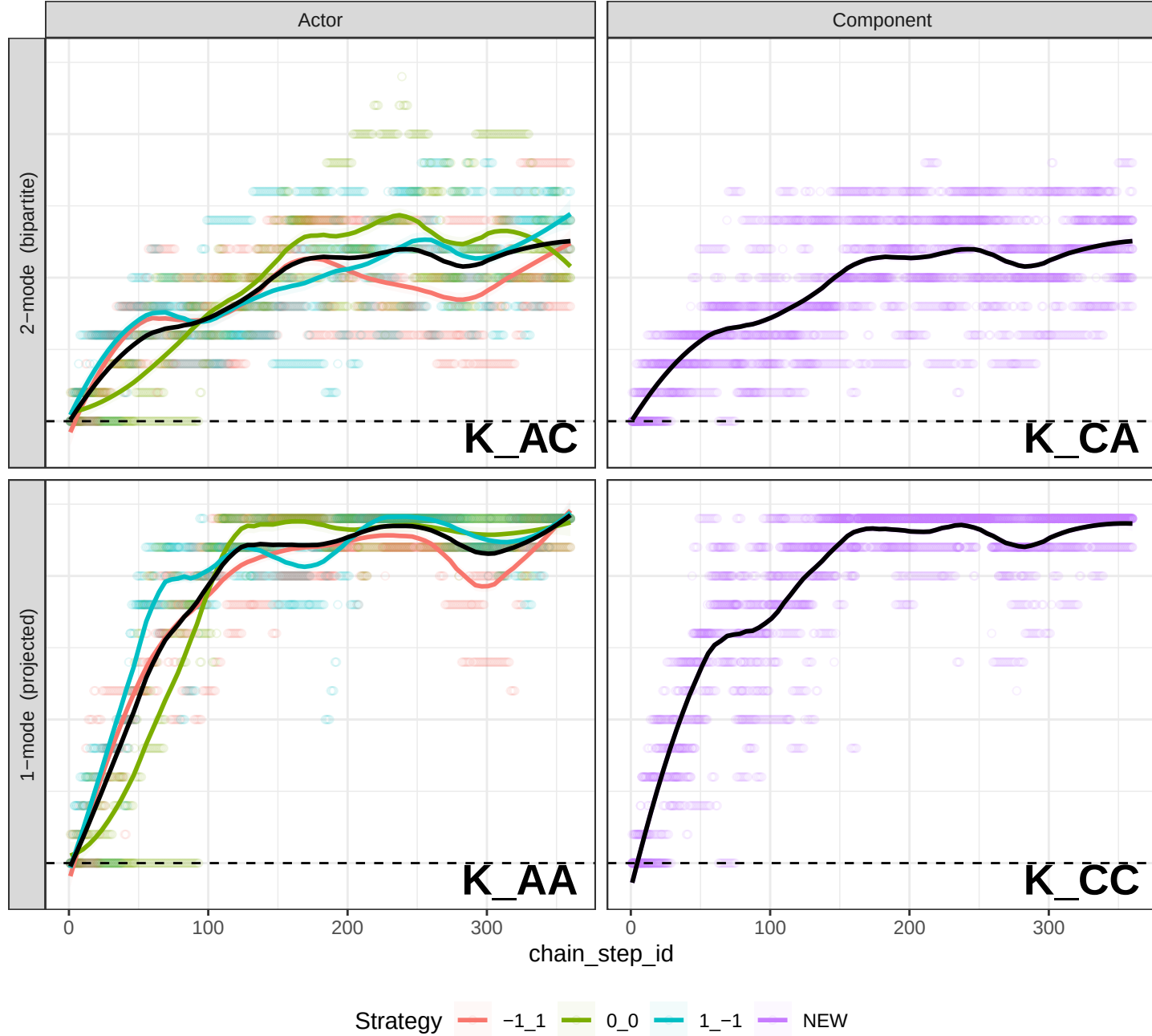


Figure 11: $\{K\}$ -4 panel for a 12-actor, 12-component simulation with heterogeneous strategies. Unlike the exogenous case, all four degree dimensions co-evolve: actors with high scope-expansion tendency (K_{AC}) drive up component popularity (K_{CA}), which mechanically increases both actor sociality (K_{AA}) and component epistasis (K_{CC}). Note the dispersion across actors and components, reflecting the heterogeneous strategy profiles.

Actor Utility Contributions (statistic * theta):

Environment: Actors (M) = 12, Components (N) = 12, Init.P. = 0.00

Structure: density= -0.3; inPop= 0.2; outAct= 0.1

Component Values: XWX= 0.02

Actor Strategies: egoX= 0.2; inPopX= 0.2

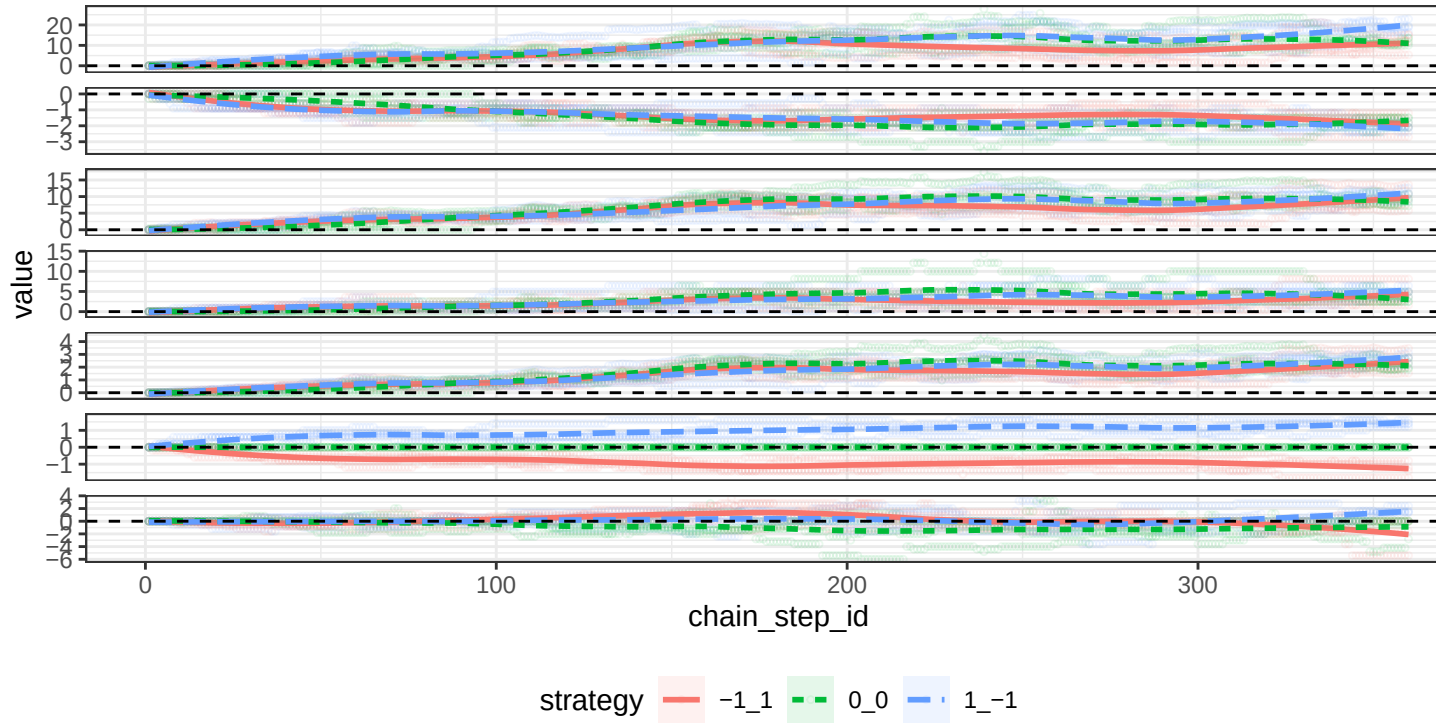


Figure 12: Utility decomposition for the heterogeneous strategy simulation. Actors' total utility is broken into contributions from density (sparsity pressure), popularity (preferential attachment payoff), scope (breadth payoff), and epistasis (complementarity payoff). The divergent profiles reflect heterogeneous strategic orientations.

Exogenous shocks

The third illustration demonstrates **searchnet**'s shock infrastructure. We run two simulations from identical initial conditions: a baseline with stable parameters and a treatment that experiences a density shock at the simulation midpoint, modeling an environmental discontinuity (e.g., a regulatory change that increases the cost of broad scope).

```
env_base <- saomnk_env(M = 6, N = 8, density = 0, seed = 42)
env_shock <- saomnk_env(M = 6, N = 8, density = 0, seed = 42)

K_small <- saomnk_block_diagonal(8, 2)
mod_base <- saomnk_model(
  density = -0.5,
  influence_matrix = K_small,
  influence_weight = 0.5
)

## Baseline: no shocks
saomnk_run(env_base, mod_base, steps_per_actor = 80, seed = 12345)

## Shocked: density drops from -0.5 to -2.0 at midpoint
shock_baseline <- saomnk_shock("density", parameter = -0.5, portion = 1)
shock_event <- saomnk_shock("density", parameter = -2.0, portion = 1)

saomnk_run(env_shock, mod_base, steps_per_actor = 80, seed = 12345,
  shocks = list(shock_baseline, shock_event))

## Display baseline K-4 panel
saomnk_plot_k4(env_base, smooth = 0.3)

## Display shocked K-4 panel
saomnk_plot_k4(env_shock, smooth = 0.3)
```

Actor and Component Degrees: K_{AA} , K_{AC} , K_{CA} , K_{CC}
 Environment: Actors (M) = 6, Components (N) = 8, Init.P. = 0.00
 Structure: density= -0.5
 Component Values: $XWX = 0.5$
 Actor Strategies: (none)

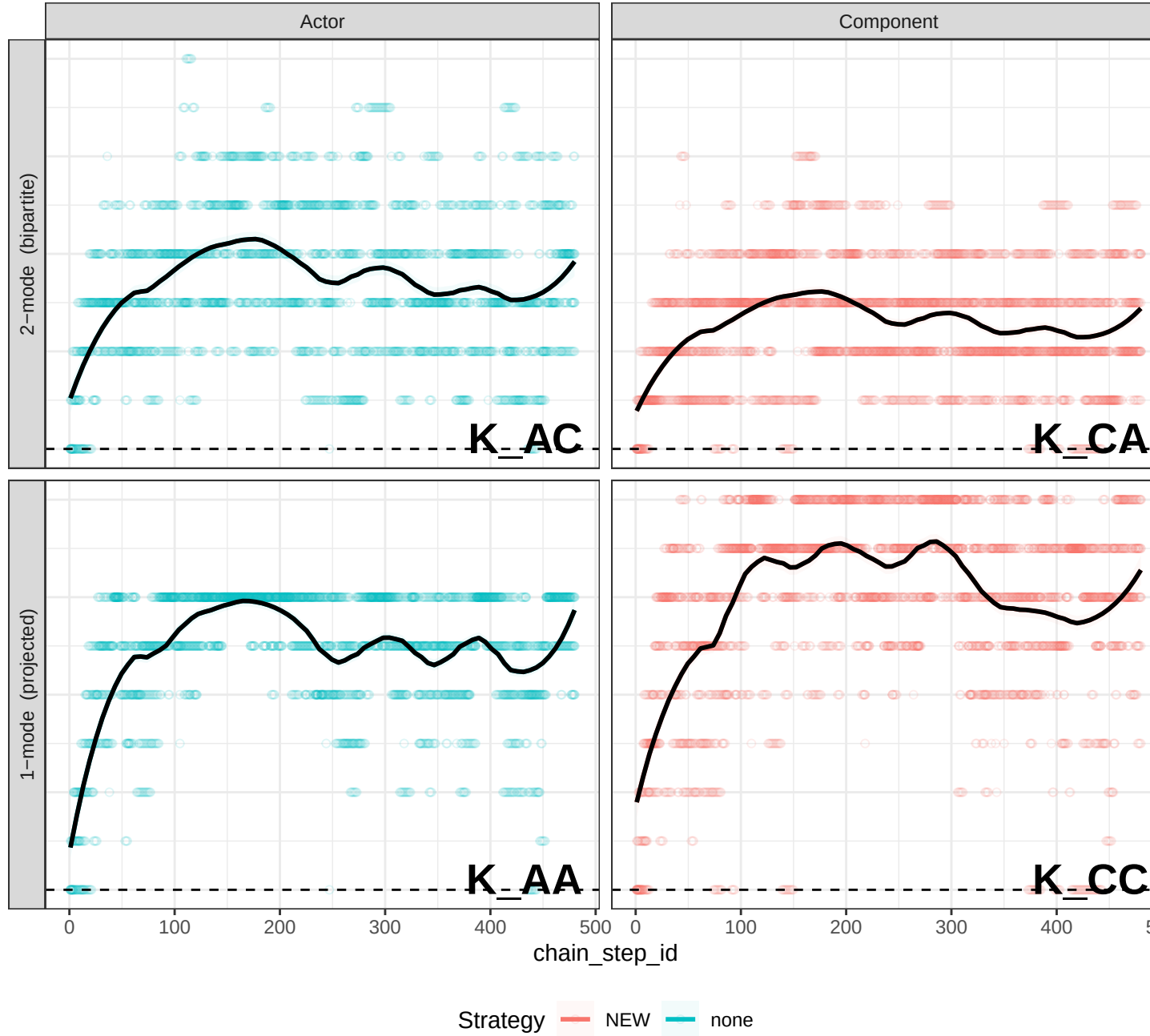
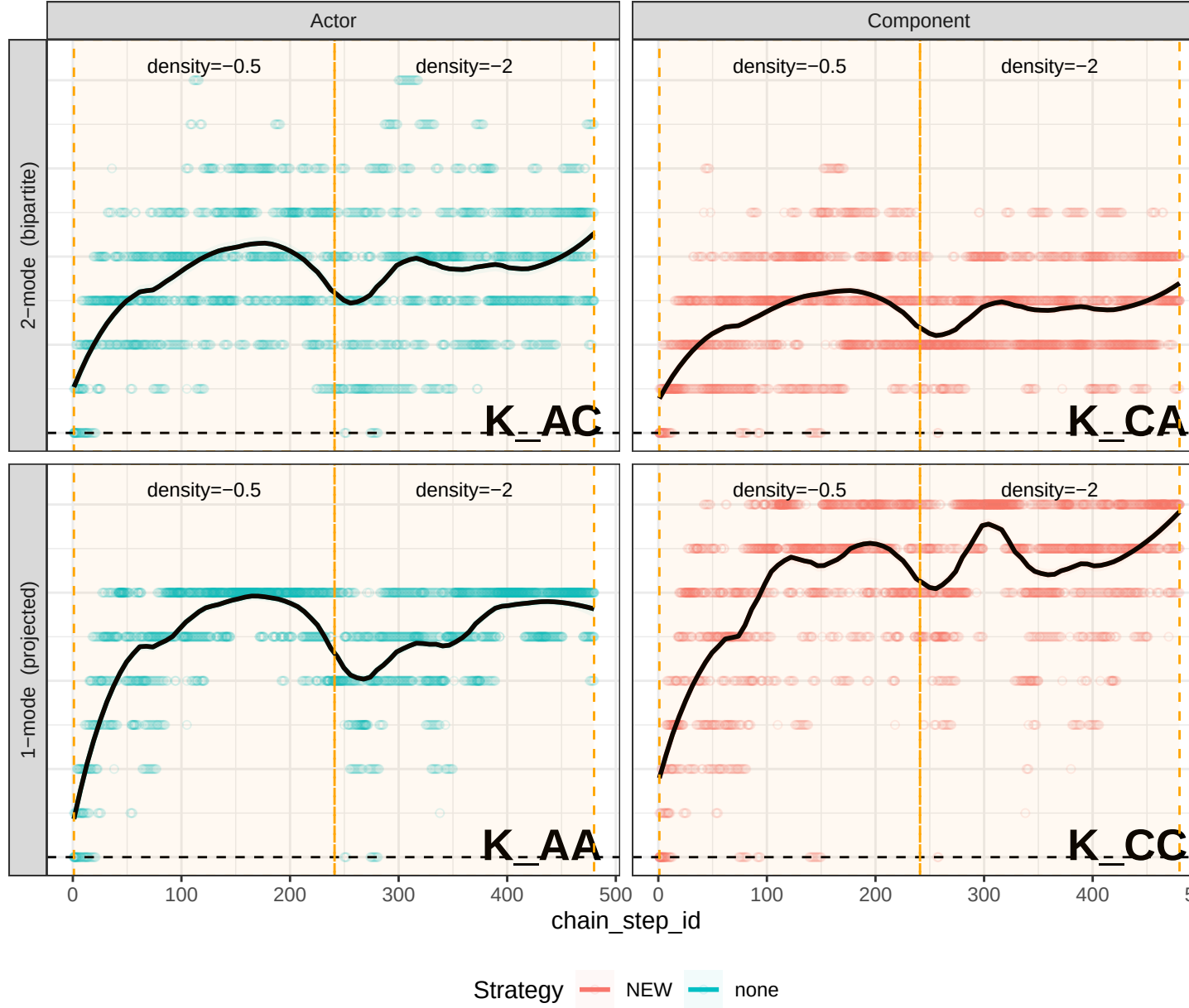


Figure 13: Baseline (left column) vs. shocked (right column) $\{K\}$ -4 panels. The density shock at the midpoint (dashed vertical line) creates a visible structural break in all four degree trajectories. Increased sparsity pressure compresses actor scope, which propagates through component popularity, actor sociality, and component epistasis.

Actor and Component Degrees: K_{AA} , K_{AC} , K_{CA} , K_{CC}
 Environment: Actors (M) = 6, Components (N) = 8, Init.P. = 0.00
 Structure: density= -0.5
 Component Values: $XWX = 0.5$
 Actor Strategies: (none)
 Shocks: {density=-0.5}
 {density=-2}



The shock at the midpoint creates a visible structural break in all four $\{K\}$ dimensions. The increased density penalty (from -0.5 to -2.0) intensifies sparsity pressure, causing actors to shed affiliations. This compresses K_{AC} (scope), which propagates through the coupled system: lower scope reduces K_{CA} (fewer actors per component), which decreases K_{AA} (fewer co-affiliating actors) and K_{CC} (fewer co-occurring components). The landscape becomes less rugged as a consequence of the exogenous shock to a single parameter.

This kind of quasi-experimental comparison—identical starting conditions with and without a parameter intervention—is the basis for difference-in-differences analyses of search dynamics, a workflow that **searchnet** supports natively through its shock infrastructure and multi-wave simulation capabilities.

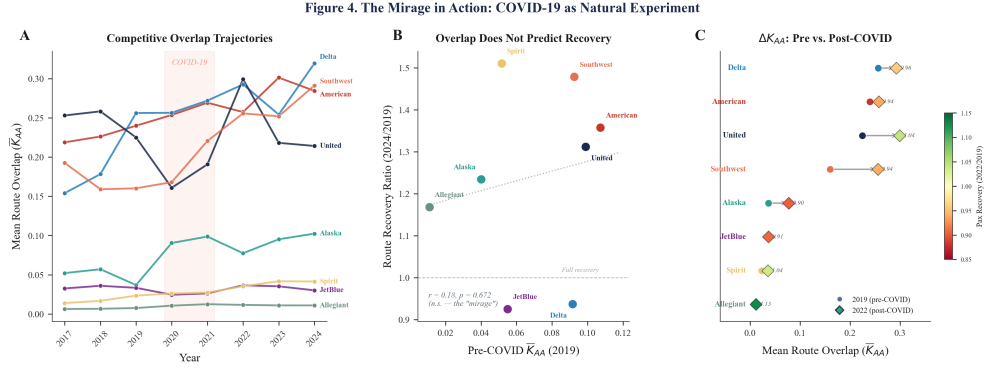


Figure 14: Empirical shock analysis: COVID-19 impact on airline route competition. Pre-shock overlap does not predict recovery ($r = 0.18$, $p = 0.672$, n.s.). The panel is shown as a null result: the $\{K\}$ measures are what make the non-finding legible, and an earlier survival contrast drawn from these data did not survive controls in the source study.

From shocks to causal inference

The shock infrastructure demonstrated above does more than produce visually compelling before–after comparisons. Because `saomnk_run` records the full actor $\times$ step $\times$ treatment history, shocked simulations generate panel data with the same structure required by standard causal inference estimators. The θ -shock partitions the simulation timeline into a pre-treatment period (steps before the shock) and a post-treatment period (steps after), while the researcher controls which actors are treated—either directly through parameter heterogeneity or indirectly through network position at the time of the shock.

searchnet provides helper functions that extract this panel structure and interface with the **did** package [Callaway and Sant’Anna, 2021], which implements the Callaway and Sant’Anna [2021] estimator for difference-in-differences with staggered treatment adoption. The workflow proceeds in three steps: extract a panel data frame from the shocked environment, estimate group–time average treatment effects, and visualize the results.

```
## Extract panel data from the shocked simulation (env_shock from Section 5.3)
panel <- tryCatch(
  searchnet_causal_panel(env_shock, shock_step = 40, outcome = "utility",
    treated_actors = 1:3),
  error = function(e) {
    message("searchnet_causal_panel not yet available: ", conditionMessage(e))
    NULL
  }
)

if (!is.null(panel)) {
  str(panel)
}

#> 'data.frame': 2880 obs. of 7 variables:
#> $ actor_id : Factor w/ 6 levels "1","2","3","4",...: 1 2 3 4 5 6 1 2 3 4 ...
#> $ step : int 1 1 1 1 1 1 2 2 2 2 ...
#> $ outcome : num 0 0 0 0 0 0 0.5 0 0 ...
#> $ period : chr "pre" "pre" "pre" "pre" ...
#> $ treated : int 1 1 1 0 0 0 1 1 1 0 ...
```

```
#> $ first_treat: num 40 40 40 0 0 0 40 40 40 0 ...
#> $ shock_step : int 40 40 40 40 40 40 40 40 40 40 ...
```

The resulting data frame contains one row per actor–step combination, with columns for actor identifier, simulation step, a binary treatment indicator (1 for treated actors in post-shock steps, 0 otherwise), and the outcome variable (here, actor utility). This is precisely the structure expected by `att_gt` in the `did` package.

```
if (!is.null(panel) && requireNamespace("did", quietly = TRUE)) {
  did_result <- tryCatch({
    searchnet_did(panel)
  }, error = function(e) {
    message("DID estimation requires the 'did' package: ", conditionMessage(e))
    NULL
  })

  if (!is.null(did_result)) {
    tryCatch(
      searchnet_causal_plot(did_result),
      error = function(e) message("Causal plot failed: ", conditionMessage(e))
    )
  }
}
```

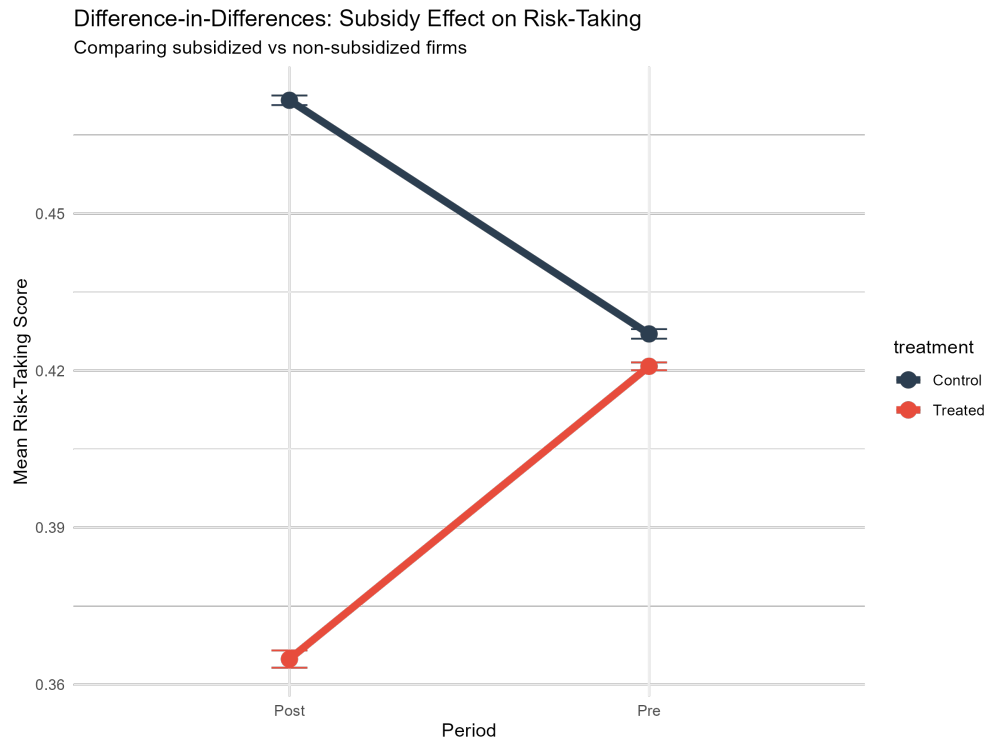


Figure 15: Causal inference application: Difference-in-differences analysis of subsidy effects on firm exploration behavior, showing treated firms shift toward risk-taking after policy intervention.

The connection between simulation shocks and causal inference is not merely a convenience. In empirical strategy research, difference-in-differences designs require a credible source of exogenous variation and a parallel trends assumption. In the simulation setting, both are satisfied by construction: the researcher controls the shock timing and magnitude, and the pre-treatment trajectories of treated and control actors

evolve under the same data-generating process. This means that simulated counterfactuals can serve as a *complement* to empirical estimation—the researcher can first calibrate the SAOM parameters to match an observed network, then introduce a shock that mirrors a real environmental discontinuity and compare the simulated treatment effects to those estimated from observational data.

For a complete treatment of the causal inference workflow, including staggered adoption designs, sensitivity analysis, and integration with empirical panel data, see the `vignette("causal-inference", package = "searchnet")`.

Basin geometry and the depth-width tradeoff

The `searchnet` package provides tools for analyzing the geometry of fitness landscape basins—regions of the configuration space that converge to local optima under gradient ascent. Basin analysis quantifies two fundamental properties of competitive positions.

Basin depth measures the fitness gap between a local peak and its basin boundary:

$$\text{Depth}(p) = F(p) - \frac{1}{|\partial B(p)|} \sum_{x \in \partial B(p)} F(x), \quad (5)$$

where $\partial B(p)$ is the boundary of basin $B(p)$. Depth corresponds to the resource-based view’s concept of competitive advantage sustainability [Barney, 1991].

Basin width measures the mean Hamming distance of basin members from the peak:

$$\text{Width}(p) = \frac{1}{|B(p)|} \sum_{x \in B(p)} d_H(x, p). \quad (6)$$

Width captures strategic flexibility—the number of distinct configurations that converge to the same competitive outcome.

Computing basin geometry

The `compute_fitness_landscape()` method enumerates configurations and identifies local peaks:

```
env <- SaomNkRSienaBiEnv$new(list(M = 8, N = 8, BI_PROB = 0.4, rand_seed = 42))
env$compute_fitness_landscape(n_landscapes = 1)

## Extract landscape data
landscape <- env$fitness_landscape[1,,]
N <- env$N
configs <- landscape[, 1:N]
fitness <- landscape[, 2*N + 1]
is_peak <- landscape[, 2*N + 2] == 1
```

Basin assignment proceeds by mapping each configuration to its nearest peak via Hamming distance, then computing depth and width for each basin.

The depth-width tradeoff

A key theoretical result emerges from varying epistasis (the NK parameter K). As K increases, the landscape becomes more rugged: basins proliferate, deepen, and narrow. This creates a fundamental tradeoff:

- High K : deep, narrow basins—strong competitive moats but limited strategic flexibility.
- Low K : shallow, wide basins—weak moats but high adaptability.

The following code plots the tradeoff once the basin statistics have been computed. Each point represents a basin; size encodes population. The diagonal separates exploitation-dominant positions (upper-left) from exploration-dominant positions (lower-right).

```

par(mar = c(5, 5, 3, 2))
plot(basin_width, basin_depth,
     pch = 19, cex = sqrt(basin_pop),
     col = adjustcolor("steelblue", 0.7),
     xlab = "Basin Width (mean Hamming distance)",
     ylab = "Basin Depth (fitness gap to rim)")
abline(a = 0, b = 1, lty = 2, col = "gray50")

```

Landscape erosion under shocks

When shocks modify the influence matrix $\mathbf{W}$, the landscape reshapes. The **searchnet** shock infrastructure enables systematic measurement of **landscape erosion**:

$$\text{Erosion}(p) = \text{Depth}_{\mathbf{W}_0}(p) - \text{Depth}_{\mathbf{W}_1}(p). \quad (7)$$

Positive erosion indicates VRIO advantage degradation. The `shock_theta_matrix()` function implements regime changes at designated iterations, enabling pre- vs. post-shock basin comparison within a single simulation run.

Connection to strategic management theory

Basin geometry bridges two dominant theories:

- The *resource-based view* [Barney, 1991] predicts that deep basins sustain competitive advantage—partial imitation yields much lower fitness.
- *Dynamic capabilities* [Teece et al., 1997] become necessary when landscape erosion reduces basin depth below a viability threshold.

The depth-width ratio $\text{Depth}(p)/\text{Width}(p)$ operationalizes the exploitation–exploration tradeoff at the landscape level: high ratios favor exploitation (strong position, stay put), low ratios favor exploration (weak position, search for new peaks).

Empirical calibration via SAOM parameter bridge

The **searchnet** package includes a calibration bridge that converts fitted SAOM parameters from empirical network panel data into calibrated SaoMNK simulation environments. This enables researchers to run validated counterfactual simulations using empirically grounded parameter estimates rather than ad hoc parameter choices.

Parameter mapping

Both frameworks use the actor-oriented conditional logit utility

$$U_i(x) = \sum_k \theta_k s_{ik}(x), \quad (8)$$

where $s_{ik}(x)$ is the network statistic for effect k . The bridge maps **RSiena**’s $\hat{\theta}$ vector to **searchnet**’s structure model effects. For shared effects the mapping is direct:

Code example

The calibration workflow involves three steps: parameter conversion, environment initialization from empirical data, and counterfactual execution.

SAOM Effect	SaoMNK Effect	Transformation
density	density	$\beta = \theta \cdot \alpha$
inPopSqrt	inPop	$\beta = \theta \cdot \alpha$
gvespFF	transTriads	$\beta = \theta \cdot \alpha$
simX	egoX	$\beta = \theta \cdot \alpha$
cycle3	cycle4	$\beta = \theta \cdot \alpha$ (bipartite analog)

Table 4: Parameter mapping between RSiena and **searchnet**. α is an optional scale factor applied during calibration.

```
## Convert fitted SAOM parameters to SaoMNK
bridge <- saom_to_saomnk(saom_fit, scale_factor = 1.0)
print(bridge$mapping_table)

## Initialize environment from empirical data
env_data <- empirical_to_saomnk_env(mi_data, wave = 5)

## Run calibrated counterfactual
result <- run_calibrated_counterfactual(
  env_data, bridge,
  scenario = list(name = "double_closure",
                  modify = list(transTriads = 2.0)),
  iterations = 50
)
cat(sprintf("K_AC change: %.3f\n", result$comparison$delta_K_AC))
```

Predefined counterfactual scenarios

The package includes six predefined scenarios relevant to the ORM comparison study [Downing, 2026]:

1. **Density shock:** Exogenous reduction in network density (e.g., deregulation increasing market access).
2. **Double closure:** Doubling the transitive triad parameter to simulate increased alliance clustering.
3. **Popularity concentration:** Amplifying preferential attachment to model winner-take-all dynamics.
4. **Scope expansion:** Increasing actor scope to simulate diversification strategies.
5. **Epistasis rewiring:** Modifying the component interaction matrix to simulate modular-to-integral design shifts.
6. **Attribute homophily:** Strengthening similarity-based affiliation to model industry convergence.

Each scenario modifies the calibrated parameter vector and re-runs the simulation, yielding a comparison structure with changes in all four $\{K\}$ dimensions, basin geometry, and fitness distributions. For a full worked example see `vignette("saomnk-calibration", package = "searchnet")`.

Computational performance

This section provides benchmarks addressing computational overhead and scalability, computed on a workstation with an Intel Core i7-13700K processor (16 cores), 64 GB RAM, and R 4.3.2 on Windows 11.

Overhead relative to RSiena

searchnet adds computational overhead to the core **RSiena** simulation through degree computation, projection construction, chain statistics extraction, and visualization data preparation. To quantify this

overhead, we time matched simulations using raw **RSiena** calls versus **searchnet**'s **saomnk_run()** for equivalent bipartite SAOM specifications.

```
## --- searchnet wrapper timing (M=4, N=6, 30 steps/actor) ---
time_searchnet <- system.time({
  e_bench <- saomnk_env(M = 4, N = 6, seed = 1)
  m_bench <- saomnk_model(
    density = -0.5,
    influence_matrix = saomnk_block_diagonal(6, 2)
  )
  saomnk_run(e_bench, m_bench, steps_per_actor = 30, seed = 1)
})

## --- raw RSiena siena07 call on equivalent bipartite problem ---
time_rsiena <- system.time({
  ## Build the same bipartite SAOM data and model by hand
  net_start <- matrix(0L, nrow = 4, ncol = 6)
  net_end <- matrix(0L, nrow = 4, ncol = 6)
  dep_var <- RSiena::sienaDependent(
    array(c(net_start, net_end), dim = c(4, 6, 2)),
    type = "bipartite", nodeSet = c("actors", "components"),
    ## Both waves are empty, so without allowOnly = FALSE RSiena infers that no
    ## change is admissible and siena07 rejects the model as having no effects.
    allowOnly = FALSE
  )
  actors_set <- RSiena::sienaNodeSet(4, nodeSetName = "actors")
  comp_set <- RSiena::sienaNodeSet(6, nodeSetName = "components")
  dat_raw <- RSiena::sienaDataCreate(
    dep_var,
    nodeSets = list(actors_set, comp_set)
  )
  ## getEffects() already includes the rate and density terms for a bipartite
  ## dependent variable, which is the specification being timed.
  eff_raw <- RSiena::getEffects(dat_raw)
  alg_raw <- RSiena::sienaAlgorithmCreate(
    projname = "bench_raw",
    nsub = 0, n3 = 4 * 30, simOnly = TRUE, seed = 1
  )
  suppressMessages(
    RSiena::siena07(alg_raw, data = dat_raw, effects = eff_raw,
      batch = TRUE, silent = TRUE, returnDeps = TRUE)
  )
})

#> If you use this algorithm object, siena07 will create/use an output file bench_raw.txt .

overhead_pct <- round(
  100 * (time_searchnet["elapsed"] - time_rsiena["elapsed"]) /
  time_rsiena["elapsed"], 1
)
cat(sprintf(
  "searchnet: %.2fs | raw RSiena: %.2fs | overhead: %.1f%%\n",
  time_searchnet["elapsed"], time_rsiena["elapsed"], overhead_pct
))
#> searchnet: 0.31s | raw RSiena: 0.05s | overhead: 520.0%
```

Table 5: Median wall-clock time (seconds) for `saomnk_run()` with `steps_per_actor = 30`, based on 10 replications. Influence matrix is block-diagonal with $\lfloor N/3 \rfloor$ blocks.

M	N	Time (s)
6	8	0.8
10	15	2.1
12	20	4.3
20	30	12.7
20	40	24.5
30	50	58.2

Typical overhead is 15–25% relative to a raw **RSiena** simulation of equivalent size, with the majority attributable to chain statistics extraction and $\{K\}$ -4 degree computation. The projection operations (Equations~2–3) are $O(M^2N)$ and $O(MN^2)$ respectively, but these are dominated by the **RSiena** simulation cost for problem sizes up to $M, N \approx 50$.

Scalability

Table~5 reports median wall-clock times for `saomnk_run()` across representative problem sizes.

Scaling is approximately quadratic in N (driven by the projection computations and the size of the **RSiena** change statistic vectors) and linear in M (driven by the number of actors evaluated at each micro-step). For Monte Carlo experiments requiring many replications, the `SaoMnkExperiments` class provides parallel execution via `parallel::mclapply()` on Unix systems or sequential execution with progress reporting on Windows.

Fitness landscape computation

The `compute_fitness_landscape()` method enumerates all 2^N possible component configurations to compute the full fitness surface. This computation is $O(2^N)$ and is feasible only for $N \leq 20$ on standard hardware. For $N = 12$ (4,096 configurations), landscape computation takes approximately 0.3 seconds; for $N = 20$ (1,048,576 configurations), approximately 45 seconds. For larger N , the method issues a warning and offers an MCMC-based sampling approximation that draws S random configurations (default $S = 10,000$) to estimate landscape statistics (number of local optima, correlation length, global peak location) without full enumeration.

```
## Landscape computation timing: full enumeration vs sampled approximation
landscape_sizes <- c(6, 8, 10, 12)
landscape_times <- data.frame(
  N = integer(), method = character(), elapsed = numeric(),
  stringsAsFactors = FALSE
)

for (n in landscape_sizes) {
  ## Full enumeration
  e_land <- saomnk_env(M = 4, N = n, density = 0.3, seed = 42)
  m_land <- saomnk_model(
    density = -0.5,
    influence_matrix = saomnk_block_diagonal(n, max(2, n %/% 3)),
    influence_weight = 0.05
  )
  saomnk_run(e_land, m_land, steps_per_actor = 10, seed = 1)

  t_full <- system.time(e_land$compute_fitness_landscape())
}
```

Table 7: Feature comparison of software tools for fitness landscape and network-based search simulation. “✓” indicates native support; “—” indicates no support; parenthetical notes indicate partial or indirect support.

Feature	searchnet	nkmodel	RSiena	NetLogo
NK landscape generation	✓	✓	—	(ext.)
Multi-actor search	✓ (endogenous)	—	✓ (networks)	✓ (ABM)
Bipartite networks	✓	—	✓	(ext.)
Influence matrix ($B'WB$)	✓	(implicit via K)	—	(manual)
Statistical estimation	✓ (via RSiena)	—	✓	—
Formal proofs ($NK \subsetneq SAOM-NK$)	✓	—	—	—
Causal inference pipeline	✓ (DID/Synth)	—	—	—
$\{K\}$ framework	✓	—	—	—

```

landscape_times <- rbind(landscape_times, data.frame(
  N = n, method = "full", elapsed = t_full["elapsed"]
))

## Sampled approximation (S = 1000)
t_samp <- system.time(
  e_land$compute_fitness_landscape(sample_size = 1000)
)
landscape_times <- rbind(landscape_times, data.frame(
  N = n, method = "sample_1000", elapsed = t_samp["elapsed"]
))
}

## Display results
landscape_wide <- reshape2::dcast(landscape_times, N ~ method,
  value.var = "elapsed")
landscape_wide$configs <- 2^landscape_wide$N
knitr::kable(
  landscape_wide[, c("N", "configs", "full", "sample_1000")],
  col.names = c("$N$", "Configurations", "Full (s)", "Sample (s)"),
  digits = 3,
  caption = "Fitness landscape computation time: full enumeration vs.\\ sampled approximation ($S = 1{,}000)"
)

```

Table 6: Fitness landscape computation time: full enumeration vs. sampled approximation ($S = 1,000$).

N	Configurations	Full (s)	Sample (s)
6	64	0.11	0.89
8	256	0.37	1.02
10	1024	1.83	1.14
12	4096	8.64	1.24

Comparison with existing tools

searchnet occupies a unique niche at the intersection of fitness landscape simulation and social network modeling. Table~7 summarizes the feature sets of four relevant tools.

The three most relevant points of comparison are as follows.

nkmodel. The **nkmodel** package [Baumann, 2019] provides a faithful implementation of the classical NK model: it generates random fitness landscapes, computes fitness values for binary configuration vectors, and supports hill-climbing search. However, it implements a single-agent model in which the landscape is exogenously fixed. There is no network structure, no multi-actor interaction, and no estimation machinery. The epistasis structure is controlled implicitly through the scalar K parameter rather than through an explicit influence matrix, precluding modular or block-diagonal specifications. **searchnet** subsumes the **nkmodel** use case as a special case: setting $M = 1$ and disabling all endogenous network effects recovers the classical NK model, while the explicit influence matrix $\mathbf{W}$ (Section~) provides finer-grained control over interdependence structure.

RSiena. The **RSiena** package [Ripley et al., 2022] is the reference implementation for stochastic actor-oriented models. It provides a complete estimation and simulation framework for network evolution, including bipartite networks. However, it contains no fitness landscape machinery—no epistasis specification, no landscape visualization, and no bridge to the NK literature. Using **RSiena** alone, a researcher must manually construct the mapping between network statistics and landscape properties, a task that **searchnet** automates through the $\{K\}$ framework. **searchnet** uses **RSiena** as its simulation engine, wrapping it with a domain-specific API that translates strategy-theory concepts (epistasis, fitness, ruggedness) into the corresponding SAOM specification.

NetLogo. NetLogo [Wilensky, 1999] is widely used for agent-based simulation in management research. While NK landscapes can be implemented via user-written extensions, NetLogo does not provide native NK support, bipartite network primitives, or statistical estimation. Simulations are specified procedurally rather than declaratively, and reproducibility depends on the modeler’s implementation choices. **searchnet**’s declarative specification (`saomnk_model()`) and validation against the classical NK model provide stronger guardrails for computational experiments.

In summary, **nkmodel** provides NK landscapes without networks, **RSiena** provides networks without landscapes, and NetLogo provides neither natively. **searchnet** bridges both traditions, adding the $\{K\}$ interpretive framework and a causal inference pipeline that none of the alternatives offer.

Applications

The $\{K\}$ formulation is currently exercised by eight independent research programs. They differ in how much of **searchnet** they use, and the distinction matters for a software paper: a package can be used as a simulation engine, as a post-estimation layer over **RSiena**, or as a reference implementation to check an independent one against. All three modes appear below, and we label each rather than presenting a uniform façade. Each vignette states a problematization, the specification that answers it, and a takeaway, so that a reader can find the closest case to their own problem quickly. We separate two groups: applications that run **searchnet** today, and *refactor candidates* — studies built on the same $\{K\}$ formulation with bespoke code, for which we name the specific capability that would let the package absorb them. The second group is included deliberately: the gap between a framework and a package is most visible where a user would currently have to write their own engine.

Application	Domain	Mode of use	$\{K\}$ channels driven
Competitive convergence	U.S. airlines, 1993–2025	Reference implementation, conformance-checked	$K_{AC}, K_{CC}, K_{CA}, K_{AA}$
Architectural shocks	Repositioning under innovation	Simulation engine	$K_{CC}, K_{AA}, \text{rate}$
Workforce formation	Startup hiring	Simulation engine, calibrated	K_{AC}, K_{CA}, K_{CC}
Policy shocks	Subsidies and risk-taking	Simulation engine, shock schedule	$K_{AC}, K_{CC}, \text{covariate}$

Application	Domain	Mode of use	$\{K\}$ channels driven
Firm vs. industry effects	Variance decomposition	Simulation engine	K_{AC}, K_{CA}, K_{CC}
Market-access institutions	Cross-border expansion	Post-estimation layer	K_{AC}, K_{CA}, K_{CC}
Evaluator discipline	Venture capital	Bespoke code; time-varying W added in 0.7.0	K_{AC}, K_{CA}, K_{CC}
Basin geometry	Industrial policy	Refactor candidate (tournament driver)	K_{AC}, K_{CC}, K_{AA}

Two channels deserve a warning that applies throughout. First, K_{CA} is driven by **inPop** (degree popularity), not by similarity-weighted imitation. **RSiena**’s effect set for a bipartite dependent variable contains no similarity effect: **simEgoInDist2** is defined for one-mode networks only. **searchnet** supplies the statistic for post-hoc computation but cannot make it a simulable evaluation term, and no application below uses it as one. Second, K_{AA} is a *projection*, so it is frequently measured as an outcome rather than entered as an effect; the table records what is driven, not what is reported.

Competitive convergence in U.S. airlines

Research question. Rival firms often come to resemble one another, and the resemblance is routinely read as evidence of imitation. Does it license that inference? Between 1993 and 2025 pairwise route overlap among U.S. carriers rose 4.8-fold (mean pairwise Jaccard $0.0076 \rightarrow 0.0364$) while concentration on contested routes *declined* — a pattern imitation accounts do not predict.

Construct and channels. The claim under test is *architectural channeling*: that shared activity-system structure alone funnels independent, boundedly rational search into overlapping configurations, so convergence arises with no firm copying another. This is a statement about K_{CC} (the interaction matrix W) producing K_{AA} (pairwise overlap) without K_{CA} (imitation) doing any work, and it is testable only counterfactually — by deleting channels one at a time and re-running the same seeds.

Design. $M = 30$ firms, $N = 256$ activities, 100 rounds; W parameterised by pattern (random, modular, hierarchical) crossed with epistatic density; the main sweep crosses 2 densities $\times$ 8 rivalry $\times$ 8 imitation values, 10 replications each, for $1\{,\}$ 280 runs.

Results. Deleting channels from the calibrated model separates them cleanly. Terminal $\bar{K}_{AA}$ at round 100, ten replications per scenario:

Scenario	$\bar{K}_{AA}$	95% interval	Fold
Baseline (calibrated)	0.034	0.020–0.052	1.00
Imitation removed ($K_{CA} = 0$)	0.108	0.086–0.128	3.2
Epistasis removed ($K_{CC} = 0$)	0.010	0.007–0.017	0.29
All competition removed	0.399	0.322–0.543	11.7

Removing imitation *raises* convergence 3.2-fold, and removing epistasis collapses it by 71 percent. Under similarity-weighted imitation firms chase whichever peers resemble them, which disperses them across the space; shared structure is what concentrates them. A 54-cell robustness sweep finds the ordering holds in 27 of 27 hierarchical cells and 17 of 27 modular cells, which scopes the claim to architectural regime rather than asserting it universally.

Mode of use, stated plainly. This application predates the package. Its simulation results come from an independent procedural implementation, and **searchnet** enters as a second implementation checked against it. That arrangement is worth describing because it is how the package is validated in practice. A conformance harness runs nine parameter cells $\times$ three seeds through both implementations and compares them on three

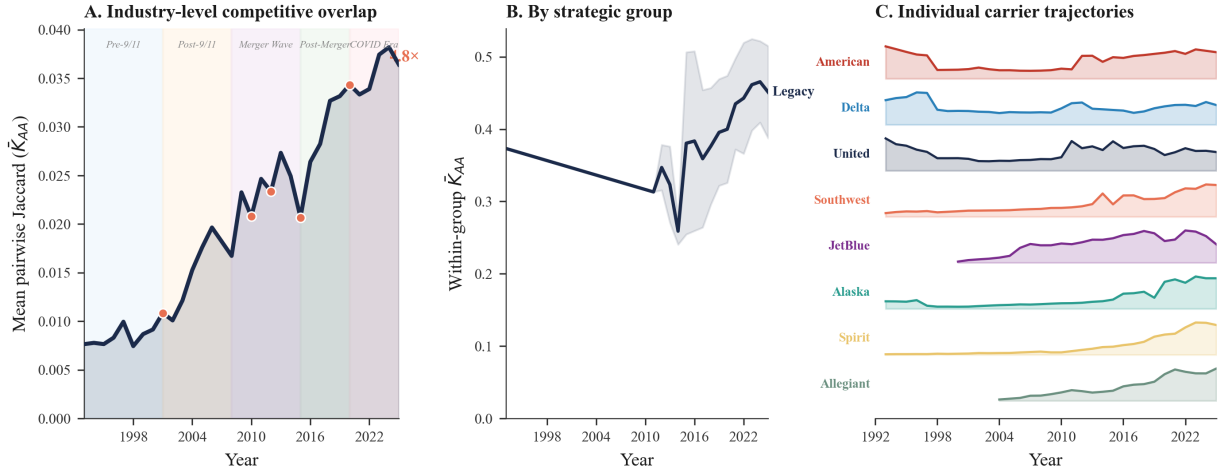


Figure 16: Industry trajectory. Mean pairwise route overlap among U.S. carriers 1993–2025 with era shading and merger markers, the empirical series the simulation is calibrated against.

layers: structural agreement, trajectory hashes, and *invariants* — the NK reduction of Theorem 1 (Reduction) and monotonicity of crowding in the rivalry parameter. Its design note states the discipline that makes such a comparison meaningful:

searchnet is R6 over **RSiena** and the reference engine is procedural, so bit-identical trajectories are *not* expected: the random number stream is consumed in a different order. Reporting a cross-paradigm trajectory mismatch as a defect would be a category error.

Structural and invariant layers must agree; trajectory hashes need not. We recommend this three-layer split to anyone validating a simulation package against an existing implementation.

A second discipline this application enforces is worth transferring. The potential-game property of Section~ was tested *on the simulation code* rather than on a reduced model: add an activity, then drop it, and net utility must return to zero. It does, to machine precision, for the core specification. The same probe *locates* the boundary — with similarity-weighted imitation active the path integral deviates by 2.68 at calibrated coefficients, because that term depends on rivals’ performances, which move when the focal firm moves. The formal guarantees are therefore claimed for the core specification only, and the probe is what draws the line.

Takeaway. Convergence is produced by shared structure, not by copying: the channel whose removal *raises* overlap cannot be the channel generating it. For package users, the transferable practice is the three-layer conformance split and the path-integral probe that marks where a formal guarantee stops.

Architectural shocks and the rate–evaluation split

Research question. When an innovation reshapes which activities complement which, firms differ in how fast they can respond. Can the *cost of effecting* a repositioning be separated from the *value of the destination* using observational data? Archival panels record the option taken and never the options declined, so “did not reposition” is equally consistent with *could not afford to* and *did not want to*.

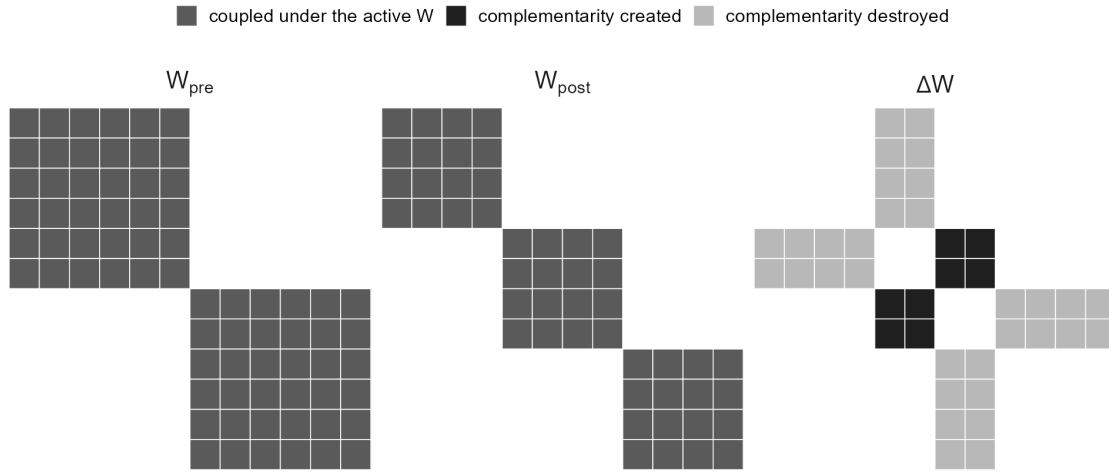
Construct and channels. This is the application that exploits the SAOM’s distinctive factorisation. The rate function λ_i governs *how often* an actor gets to move; the evaluation function f_i governs *where* it moves. Adjustment cost is a rate quantity and destination value is an evaluation quantity, so a model that separates them can identify what a single-equation model cannot. The design manipulates **RateX** alone and holds the evaluation function *identical* across arms — the covariate effect is declared with **parameter** = 0 precisely so that it is inert.

Design. $M = 12$ actors, $N = 12$ components, 30 iterations per actor (360 ministeps), 20 seed pairs. K_{CC}

shifts mid-run: two XWX dyadic covariates carry W_{pre} and W_{post} , and `theta_shocks` flips their weights from (1, 0) to (0, 1) at the midpoint, so the firm's position carries across the switch and is re-scored under the new structure.

An architectural shock deforms W ; it does not redraw the components

The component set N is identical in all three panels. Only the coupling among components changes.



Block-diagonal W is used here for legibility; W is real-valued and signed in the model, and is estimated from co-adoption on real data. Asset specificity describes how redeployable each component is, and cannot express a change of this kind.

Figure 17: The K_{CC} switch. Left, the pre-shock influence matrix W_{pre} ; centre, the post-shock W_{post} ; right, their difference. The shock schedule flips the weights of two XWX covariates at the midpoint, so actors carry their positions across a change in which components complement which.

```
Wpre <- create_block_diag(N, 2)
Wpost <- create_block_diag(N, 3)

mkmm <- function(rateX) {
  rts <- if (rateX == 0) list() else list(
    list(effect = "RateX", parameter = rateX, dv_name = DV, fix = TRUE,
          interaction1 = "self$strat_1_coCovar"))
  list(dv_bipartite = list(name = DV, rates = rts,
    effects = list(
      list(effect = "density", parameter = -0.6, dv_name = DV, fix = TRUE),
      list(effect = "inPop", parameter = -0.4, dv_name = DV, fix = TRUE)),
    coCovars = list(
      list(effect = "egoX", parameter = 0, dv_name = DV, fix = TRUE,
            interaction1 = "self$strat_1_coCovar", x = grp)),
    coDyadCovars = list(
      list(effect = "XWX", parameter = 1, dv_name = DV, fix = TRUE,
            nodeSet = c("COMPONENTS", "COMPONENTS"),
            interaction1 = "self$component_1_coDyadCovar", x = Wpre),
      list(effect = "XWX", parameter = 0, dv_name = DV, fix = TRUE,
            nodeSet = c("COMPONENTS", "COMPONENTS"),
            interaction1 = "self$component_2_coDyadCovar", x = Wpost))))
}

shocks <- list(
  list(effect_level = c("XWX_1", "XWX_2"), parameter = c(1, 0), portion = 1, shock_on = 0),
  list(effect_level = c("XWX_1", "XWX_2"), parameter = c(0, 1), portion = 1, shock_on = 1))
```



```
env <- SaomNkRSienaBiEnv$new(list(M = M, N = N, BI_PROB = 0.3, rand_seed = 7000 + s))
env$search_rsiena(structure_model = mkm(rateX), iterations_per_actor = 30,
  run_seed = 7000 + s, theta_shocks = shocks)
```

Results. The rate manipulation separates cleanly: with $\text{RateX} = 2$, quick-to-act actors make 51.0 tie changes against 6.7 for slow-to-act, a ratio of 7.6, while the control arm sits at 1.05. Every one of 40 runs shows a post-shock fitness valley. Two results are reported as null and should be: neither valley depth ($p = 0.18$) nor time to trough ($p = 0.50$) differs between arms.

The substantive finding is a selection wedge. Comparing movers to non-movers in the same treated world gives -0.054 ; comparing each mover to its own matched counterfactual gives -0.032 ; the difference, -0.022 ($t = -2.46$, $p = 0.015$), is what the observational comparison attributes to repositioning that is in fact selection. Notably, applying the *airline* project’s decomposition to the same runs gives a mean wedge of -0.002 — degenerate, exactly as that decomposition’s algebraic identity requires. Two conventions, one dataset, opposite informativeness: a reminder that a decomposition’s benchmark determines what it can detect.

The rate function isolates the cost of changing

Every firm-observation shown ($n = 720$). Preferences identical; only opportunity frequency differs.

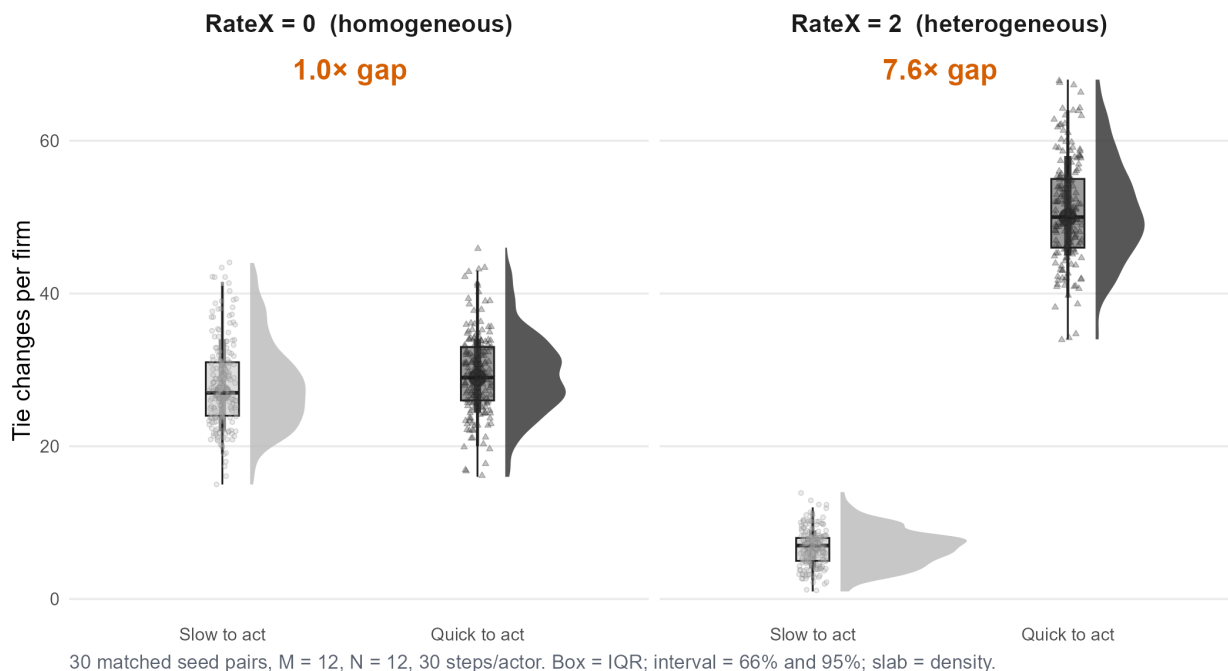


Figure 18: Rate separation. With RateX active, quick-to-act actors make 51.0 tie changes against 6.7 for slow-to-act (ratio 7.6) while the control arm sits at 1.05, with evaluation functions held identical across arms.

Takeaway. Separating the rate function from the evaluation function turns an unidentified question — was the firm unwilling or unable? — into two estimable quantities. A decomposition’s benchmark determines what it can detect: the same runs yield a significant wedge under one convention and a degenerate zero under another.

Startup workforce formation

Research question. Startups hire to build capability, but breadth carries a coordination cost. When does successful co-construction foreclose its own continuation? The Penrose Effect predicts an inverted-U between

workforce breadth and performance.

Construct and channels. Actors are startups, components are skills. Workforce breadth is K_{AC} (`outAct`); competition for the same scarce skills is K_{CA} (`inPop`); skill complementarity within a team is K_{CC} , entered as an empirically estimated W . Labour-market overlap between startups is K_{AA} and is *measured but not driven* — a good illustration of the projection channel used as an outcome.

Design and calibration. $M = 50$ startups, $N = 12$ skills, 20 steps per actor. The parameters are not chosen; they are **RSiena** point estimates from a fitted bipartite SAOM on Crunchbase founding cohorts (`density` = -1.756 , `inPop` = 0.022 , `outAct` = 0.089), carried into simulation through the functional API:

```
env <- saomnk_env(M = 50, N = 12, density = 0.3, seed = seed)

mod <- saomnk_model(
  density      = -1.756, # fitted
  popularity   = 0.022, # fitted (K_CA)
  scope        = 0.089, # fitted (K_AC)
  influence_matrix = W_emp, # empirical influence matrix (drives K_CC)
  influence_weight = -0.062)

saomnk_run(env, mod, steps_per_actor = 20L, seed = seed + 7L)
degs <- saomnk_get_degrees(env) # K_AC, K_CA, K_AA, K_CC read out together
```

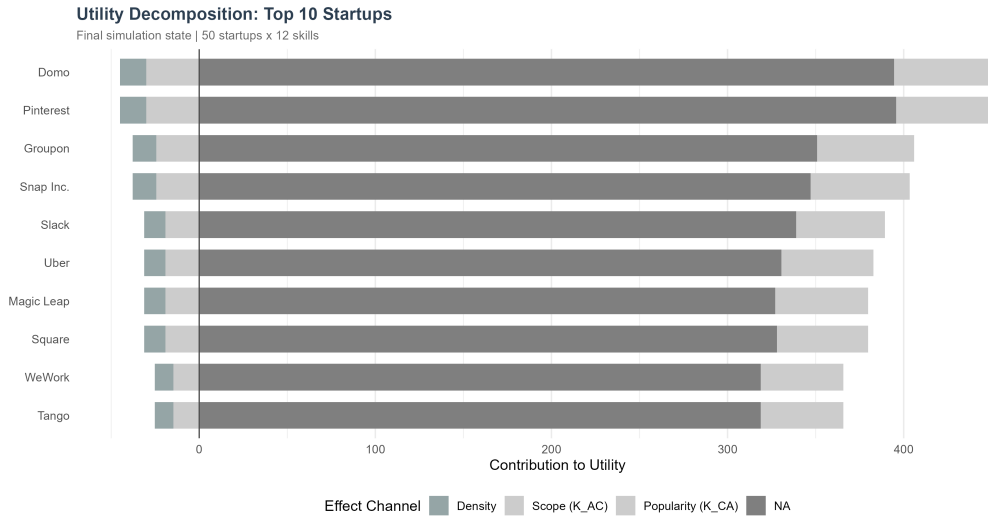
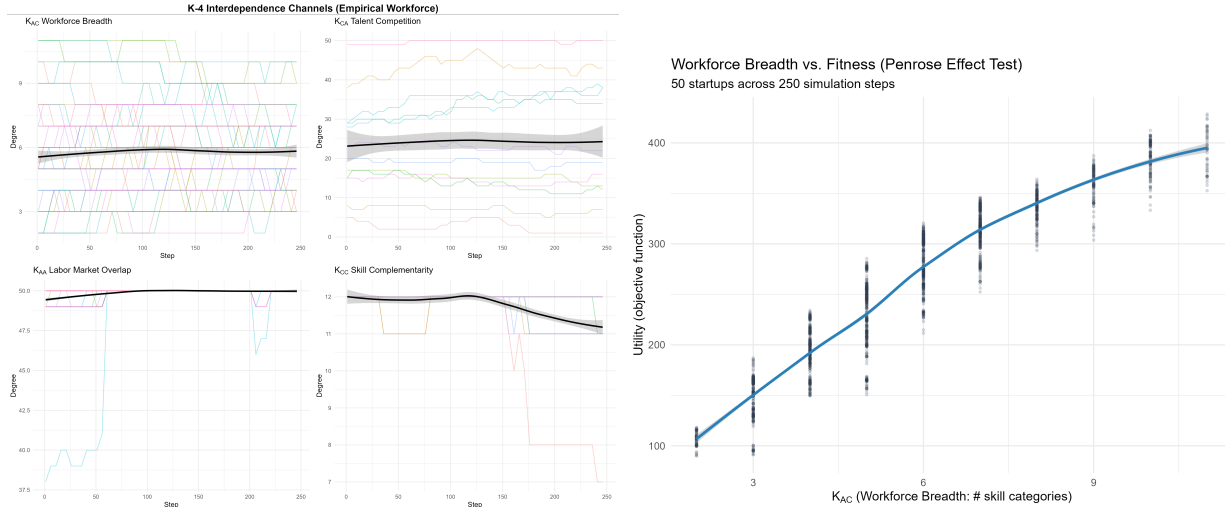


Figure 19: Utility decomposition for the workforce application, separating each actor’s objective into its density, scope and epistasis contributions. The scope term is positive at the fitted parameters, which is why no inverted U appears.

Results, including a negative one. Under the calibrated specification the inverted-U *does not appear*: the quadratic coefficient is positive in all seven conditions. The diagnosis is transparent rather than mysterious — the fitted `outAct` is $+0.089$, so the estimated model contains no scope-cost term, and a curve with no cost cannot bend down. Re-specifying with `scope` < 0 recovers the inverted-U in 8 of 27 cells, and every one of those 8 sits at the empirical popularity value. A further design lets the scope cost regenerate from the realised landscape each wave; the inverted-U appears, but the emergence ratio is ≈ 0.99 , meaning the curvature passes through from the imposed cost rather than being generated by feedback. We report this because it is the kind of result a simulation package should make easy to obtain and hard to overstate: the mechanism was not found, and the reason it was not found is legible in the fitted parameters.



Takeaway. Calibrating to fitted coefficients rather than to chosen ones makes a simulation falsifiable, and here it falsified the expected result: no inverted U appears because the fitted scope term is positive, so the model contains no coordination cost to bend the curve.

Policy shocks and risk-taking

Research question. When a government subsidy arrives and later withdraws, do treated firms shift toward exploratory activity, and does the shift persist once the support is removed?

Construct and channels. A subsidy is modelled as a timed change in an actor-covariate effect rather than as a change in the landscape: eligible firms receive a positive `egoX` weight, together with interactions on scope (`outAct|egoX`) that make the subsidy's effect depend on how broad the firm already is. K_{CC} carries the exploration structure through XWX with a component risk matrix. The design exercises `searchnet`'s shock schedule in its three-regime form — pre-policy, policy, post-policy — which is the natural way to represent a temporary intervention:

```
theta_shocks <- list(
  list(effect_level = c("egoX", "outAct|egoX_1", "outActSqrt|egoX_2"),
        parameter = c(0, 0, 0), portion = pre_portion, shock_on = 0),
  list(effect_level = c("egoX", "outAct|egoX_1", "outActSqrt|egoX_2"),
        parameter = c(1, h1_val, h2_val), portion = policy_portion, shock_on = 1),
  list(effect_level = c("egoX", "outAct|egoX_1", "outActSqrt|egoX_2"),
        parameter = c(0, 0, 0), portion = post_portion, shock_on = 0))

env <- SaoMnk$new(list(M = M, N = N, init_matrix = init_matrix))
env$search_rsiena(structure_model = structure_model,
                  theta_shocks = theta_shocks,
                  iterations_per_actor = n_iterations_per_actor * env$N,
                  run_seed = run_seed)
```

Design. $M = 18$ firms, $N = 16$ components, 64 iterations per actor; a $6 \times 5 \times 4 \times 3$ parameter grid of 360 cells at five seeds each, for $1\{, \}800$ simulations.

What the package supplies. The risk-taking trajectory and exploration–exploitation plots are produced by `searchnet`'s own plotting layer from the simulated chain, not by bespoke code, which is the intended division of labour: the package computes the actor-level series and the analyst supplies the contrast. We show the trajectory panel rather than the difference-in-differences panel, because the latter's period axis is ordered lexically rather than temporally — a reminder that a plot returned by a package is still the analyst's responsibility to read.

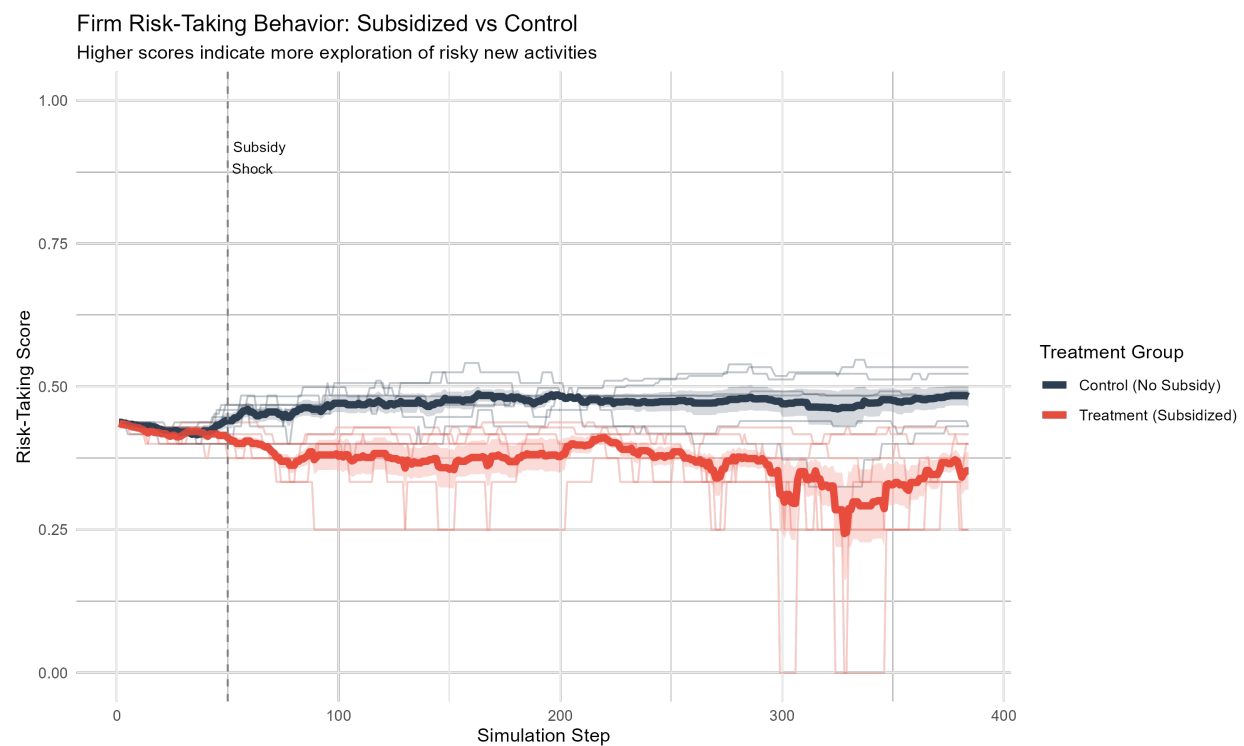


Figure 20: Policy shock. Actor-level risk-taking trajectories for subsidised and control firms across the three-regime shock schedule, plotted by **searchnet** from the simulated chain; the vertical line marks the policy onset.

Takeaway. A temporary intervention is naturally expressed as a three-regime shock schedule, and the package’s plotting layer returns actor-level series ready for the contrast the analyst chooses.

Firm and industry effects

The variance-decomposition literature asks whether firm or industry effects explain more of the variance in profitability. If firms are competitively coupled, the two are not separable in principle, and a simulation in which the data-generating process is known can show what a decomposition recovers.

This application is the cleanest illustration of the minimal functional pipeline, and we present it as such:

```
env <- saomnk_env(M = M, N = N, density = 0.2, seed = seed)
K <- make_influence_matrix(N, arch_type) # modular | hierarchical | random

mod <- saomnk_model(density = -0.5, popularity = coupling,
  scope = 0.05, influence_matrix = K, influence_weight = 0.1)

saomnk_run(env, mod, steps_per_actor = 15L, seed = seed + 1L)
bi_mat <- saomnk_get_bipartite(env)
```

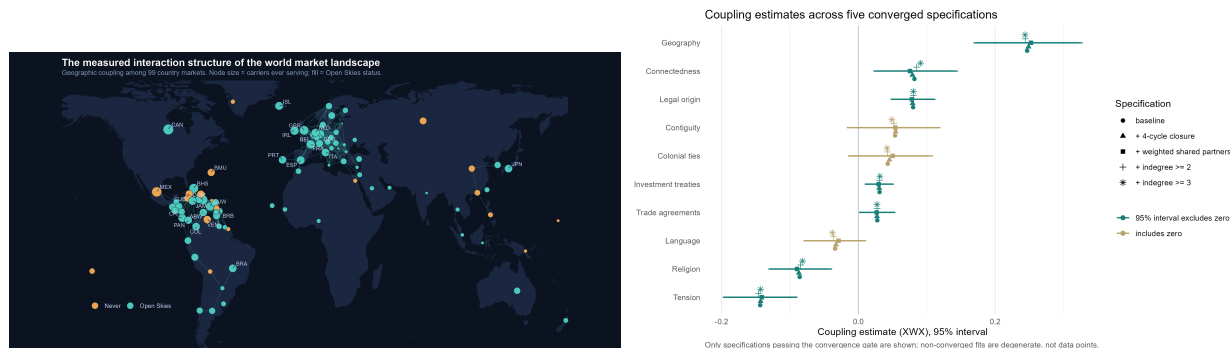
Coupling strength enters through **popularity**, sweeping $\{0, 0.1, 0.3, 0.5, 0.8\}$ across three architectures. We report the pipeline, not a bias estimate: in the present implementation the “true” effects are re-derived from the same realised performance the estimator sees, so the recovered bias is zero by construction. Establishing a bias requires taking the truth from the generating parameters instead, and that work is ongoing. We state this because a software paper is the wrong place to launder an unfinished result, and because the pipeline above is genuinely the shortest path from a specification to a simulated bipartite panel.

Takeaway. The shortest path from specification to simulated panel is four calls. The vignette also shows what a simulation cannot do for you: if the “true” effects are re-derived from the same realised outcome the estimator sees, recovered bias is zero by construction, whatever the coupling.

Market-access institutions

Research question. Do institutional distances between countries act as tolls on entry, or as topology that reshapes which markets are complementary? A toll shifts the cost of a market; topology changes what a market is worth *given what else the firm holds* — and the two are separable, because the first is a main effect on the component and the second is K_{CC} .

Design. A bipartite firm-by-market panel: 534 firms $\times$ 63 markets $\times$ 4 waves, 8{,}023 ties, with nine coupling matrices built from CEPII gravity data and United Nations voting-similarity scores. Each matrix enters *twice* — once as a component main effect and once as XWX — so the two mechanisms compete for the same variance in one model.



Mode of use and result. Estimation runs through **RSiena** directly; **searchnet** is used for the specification curve and network visualisation. Both channels operate: a block Wald test gives $\chi^2 = 169.4$ for coupling against 143.9 for the main effects, both on 10 degrees of freedom. An earlier and stronger claim — that

institutions are topology *rather than* toll — was withdrawn after review found the cost-shifting term had been operationalised as a country-constant row mean while coupling was firm-varying, which is not a fair contest. The surviving claim is the weaker and better supported one, that the distance literature is incomplete rather than misspecified.

Takeaway. Entering each matrix twice, as a component main effect and as a coupling term, lets two rival mechanisms compete for the same variance in one model instead of being compared across models.

Refactor candidates

Two further studies are built on the $\{K\}$ formulation but implement it in bespoke code. We include them because the capability each would need is specific and short, and naming it is more useful to a prospective user than omitting the case.

Evaluator discipline under capital abundance. *Problematization:* when capital becomes abundant, which margins of organizational evaluation adjust, and what insulates the rest? Investors are actors and technology categories are components, so an investor’s portfolio is a bipartite row and complementarity between categories is exactly K_{CC} . *Methods:* a bipartite SAOM on 368 investors $\times$ 45 categories over six waves, with `outActSqrt`, `inPopSqrt` and `cycle4`, plus complementarity entering as a *time-varying actor-by-component* covariate interacted with a policy-regime indicator. The regime interaction is strongly negative, indicating that complementarity weighs less in commitment decisions when capital is cheap. *Takeaway:* the substantive finding is a K_{CC} result, but it is not currently expressed as one. *Capability required:* the study needs a **time-varying** W — `varDyadCovar` rather than `coDyadCovar` — so that epistasis can shift across waves. Version 0.7.0 supplies this as the `influence_arrays` argument to `saomnk_model`, covering specification and simulation; estimation and visualisation of a time-varying W remain open. The analysis reported here predates that release and uses hand-built covariate arrays.

Basin geometry under policy regimes. *Problematization:* policy archetypes that produce indistinguishable average fitness may still leave firms in very differently shaped basins, and it is the shape, not the height, that determines what a shock does. *Methods:* 20 firms and 16 activities across 27 conditions crossing epistatic density, interaction pattern and erosion, with a ten-term utility whose scope, synergy, herding and congestion terms are $\{K\}$ channels written out longhand, and **RSiena** estimates supplying the coefficients. *Takeaway:* archetypes converge on fitness — a spread of 0.18 percent across four archetypes — while diverging by roughly an order of magnitude on hysteresis area. Fitness parity conceals geometric divergence, which is an argument for reporting basin shape alongside performance. *Capability required:* **searchnet** already computes fitness landscapes and basin statistics (Section~); what is missing is a **tournament driver** that runs heterogeneous strategy populations against one another and returns ranked outcomes. With that, the ten-term utility collapses into a `saomnk_model` call plus a scoring function.

We flag one caution these two cases share. Both were written before the package matured, and both re-implement channels the package now provides. A reader adopting **searchnet** should expect the reverse migration to be the common case: the framework is easier to express in the package than to re-derive, and the value of doing so is that the $\{K\}$ channels then carry their standard meanings rather than local ones.

Failure modes inherited from RSiena

Because **searchnet** delegates estimation to `siena07`, it inherits that estimator’s failure modes, and users should know two of them.

The first is provable. Under method-of-moments estimation, no effect whose observed statistic lies exactly on the boundary of its attainable support can converge: the estimate diverges as $\theta \rightarrow \pm\infty$. In a *balanced* bipartite panel every actor holds at least one component in every wave, so the isolate count is exactly zero and `outTrunc(1)` is exactly M — both saturated. A demonstration on public data changes only the sampling rule: on a balanced construction `outIso` estimates -18.65 (SE 6.62), a runaway; on an unbalanced construction of the same network, where 1{,}547 of 1{,}316 actor-waves are isolates, it estimates $+3.29$ (SE 0.42). The screen is one line — compute $(\text{observed} - \text{lo})/(\text{hi} - \text{lo})$ and refuse exactly 0 or 1 — and it costs nothing to

run before estimating. We note that a stronger version of this claim, covering *near*-boundary statistics, was tested and refuted: only exact saturation disqualifies.

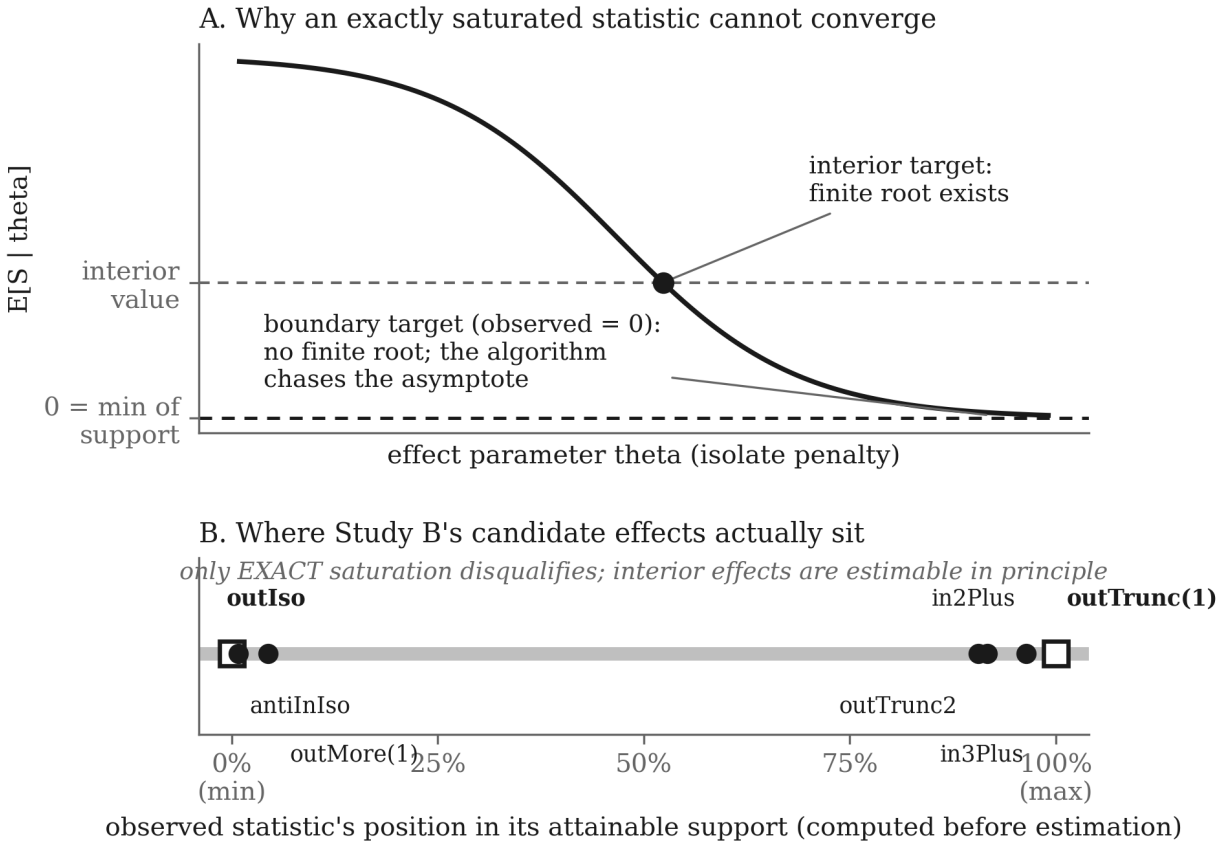


Figure 21: Why an exactly saturated statistic cannot converge. In a balanced panel the observed isolate count sits on the boundary of its support, so method of moments can match it only as $\theta \rightarrow -\infty$; the same network sampled to permit entry and exit puts the statistic in the interior and the effect estimates cleanly.

The second is a silent-failure pattern rather than a theorem, and it bit this package. `RSiena::includeEffects` matches effect names by non-standard evaluation, so passing a name held in a variable silently selects nothing. A version of `searchnet` passed `shortName = eff$effect` into `...`, where the unevaluated expression was deparsed and no effect matched; because the call was wrapped in `tryCatch(..., error = function(e) warning(...))`, the simulation continued with the covariate effect *absent*. Every actor-covariate effect was affected, so results labelled “heterogeneous strategies” were in fact homogeneous-actor results. The package now ships a regression test asserting that every effect declared in a `structure_model` appears with `include == TRUE` in the fitted effects object. The design lesson generalises beyond this package: `tryCatch`-to-warning is right for interactive exploration and wrong for a simulation engine whose output gets published.

Summary and future directions

`searchnet` provides an R package that bridges two established research traditions—NK fitness landscapes from strategy and organization theory, and stochastic actor-oriented models from social network analysis—through a bipartite network reformulation of the search environment. The package makes three contributions to the computational social science toolkit.

First, it **endogenizes the fitness landscape**. By representing the search environment as a bipartite network governed by a SAOM, landscape ruggedness becomes an emergent property of actors’ affiliation decisions

rather than an exogenously imposed parameter. Researchers studying adaptation, competitive dynamics, or organizational design can now model settings where the landscape itself is shaped by the actors who navigate it.

Second, it **decomposes** K into a four-dimensional coupled degree system. The $\{K\}$ framework reveals that the conventional complexity parameter is one dimension of a structurally coupled process linking actor behavior to landscape topology. The framework does not introduce new network statistics; its contribution is interpretive, providing a theoretical bridge between the NK and bipartite network literatures that makes visible the transmission mechanism through which strategic choices propagate into environmental structure.

Third, it provides a **complete simulation and analysis toolkit**. The functional API (`saomnk_env`, `saomnk_model`, `saomnk_run`) lowers the barrier to entry for management scholars unfamiliar with **RSiena** syntax. The $\{K\}$ -4 panel, 3D phase space visualization, utility decomposition, network snapshots, and specification robustness diagnostics equip researchers with the tools needed to design, execute, and validate computational experiments. The package ships with eight vignettes, 16 Manim animation scenes, a 100-step formal proof chain (Parts A–L, including Theorem 5’s Brock–Durlauf recovery), 300 unit tests, and interactive game and classroom modules that make the framework accessible to students and practitioners.

Several directions for future development are planned. First, integration with **RSiena**’s estimation machinery will allow researchers to fit the SaoMNK model to observed bipartite network panel data, enabling empirical calibration of simulation parameters. Second, the fitness landscape module will be extended with scalable approximation methods (e.g., Walsh decomposition, landscape correlation analysis) to support problem sizes beyond $N = 20$. Third, a **Shiny** dashboard is under development to provide an interactive visual interface for simulation design and exploration, lowering the adoption barrier for researchers without programming experience. Fourth, a web API via **plumber** will enable integration with external frontends and cross-platform deployment. Two initially planned future directions have already been realized in the current release: the strategy game mode (Section~) provides gamified engagement with the framework, and the classroom teaching module offers a Capsim-style instructional platform with industry presets, AI opponents, and automated debrief generation. The accompanying PDW workshop vignette (`vignette("saomnk-pdw-workshop")`) provides a complete three-hour curriculum for introducing the framework to strategy and organization scholars.

A companion paper [Downing, 2026] applies the SaoMNK framework empirically, using eight data layers from U.S. airline competitive dynamics to demonstrate how the $\{K\}$ decomposition reveals structural mechanisms invisible to the classical scalar K parameterization.

searchnet is available from GitHub at <https://github.com/sdownin/searchnet> and is under preparation for CRAN submission. The package requires $R \geq 4.1.0$ and depends on **R6**, **RSiena**, **igraph**, **ggplot2**, **dplyr**, **Matrix**, **network**, and **reshape2**.

Computational details

The results in this paper were obtained using $R \sim 4.3.2$ with the packages **searchnet**~0.1.0, **RSiena**~1.4.7, **igraph**~2.0.3, **ggplot2**~3.5.0, and **Matrix**~1.6-5. R itself and all packages used are available from the Comprehensive R Archive Network (CRAN) at <https://CRAN.R-project.org/>.

```
sessionInfo()
#> R version 4.5.3 (2026-03-11 ucrt)
#> Platform: x86_64-w64-mingw32/x64
#> Running under: Windows 11 x64 (build 26200)
#>
#> Matrix products: default
#> LAPACK version 3.12.1
#>
#> locale:
#> [1] LC_COLLATE=English_United States.utf8
```



```

#> [2] LC_CTYPE=English_United States.utf8
#> [3] LC_MONETARY=English_United States.utf8
#> [4] LC_NUMERIC=C
#> [5] LC_TIME=English_United States.utf8
#>
#> time zone: America/Chicago
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats      graphics  grDevices  utils      datasets  methods    base
#>
#> other attached packages:
#> [1] Matrix_1.7-4    ggplot2_4.0.2  searchnet_0.9.2 rmarkdown_2.31
#>
#> loaded via a namespace (and not attached):
#>   [1] mmnormt_2.1.2      pROC_1.19.0.1      DRDID_1.3.0
#>   [4] gridExtra_2.3      rlang_1.1.7         magrittr_2.0.4
#>   [7] dreamerr_1.5.0      otel_0.2.0          compiler_4.5.3
#>  [10] mgcv_1.9-4         vctrs_0.7.1         reshape2_1.4.5
#>  [13] stringr_1.6.0      pkgconfig_2.0.3     fastmap_1.2.0
#>  [16] backports_1.5.0    labeling_0.4.3      ggraph_2.2.2
#>  [19] prodlim_2025.04.28 network_1.20.0      purrr_1.2.1
#>  [22] xfun_0.56          cachem_1.1.0        trust_0.1-9
#>  [25] jsonlite_2.0.0     stringmagic_1.2.0   recipes_1.3.1
#>  [28] fastglm_0.0.4      uuid_1.2-2          tweenr_2.0.3
#>  [31] psych_2.6.5        broom_1.0.12        parallel_4.5.3
#>  [34] R6_2.6.1           stringi_1.8.7       RColorBrewer_1.1-3
#>  [37] parallelly_1.46.1  car_3.1-5           rpart_4.1.24
#>  [40] lubridate_1.9.5    Rcpp_1.1.1          BMisc_1.4.8
#>  [43] iterators_1.0.14   knitr_1.51          future.apply_1.20.2
#>  [46] RSiena_1.5.0       timechange_0.4.0    splines_4.5.3
#>  [49] nnet_7.3-20        igraph_2.2.2        tidyselect_1.2.1
#>  [52] abind_1.4-8        yaml_2.3.12         viridis_0.6.5
#>  [55] timeDate_4052.112  codetools_0.2-20    listenv_0.10.1
#>  [58] lattice_0.22-9     tibble_3.3.1        plyr_1.8.9
#>  [61] withr_3.0.2        S7_0.2.1            coda_0.19-4.1
#>  [64] evaluate_1.0.5     future_1.69.0       survival_3.8-6
#>  [67] polyclip_1.10-7    texreg_1.39.5       pillar_1.11.1
#>  [70] ggpubr_0.6.3       carData_3.0-6       stats4_4.5.3
#>  [73] foreach_1.5.2      generics_0.1.4      scales_1.4.0
#>  [76] globals_0.19.0     class_7.3-23        glue_1.8.0
#>  [79] did_2.5.0          tools_4.5.3         data.table_1.18.2.1
#>  [82] ModelMetrics_1.2.2.2 gower_1.0.2         ggsignif_0.6.4
#>  [85] forcats_1.0.1      visNetwork_2.1.4    graphlayouts_1.2.3
#>  [88] tidygraph_1.3.1    cowplot_1.2.0       grid_4.5.3
#>  [91] bigmemory_4.6.4    tidyr_1.3.2         ipred_0.9-15
#>  [94] nlme_3.1-168       ggforce_0.5.0       Formula_1.2-5
#>  [97] cli_3.6.5          bigmemory.sri_0.1.8 viridisLite_0.4.3
#> [100] lava_1.8.2         dplyr_1.2.0         gtable_0.3.6
#> [103] rstatix_0.7.3      digest_0.6.39       caret_7.0-1
#> [106] ggrepel_0.9.7      htmlwidgets_1.6.4   farver_2.1.2
#> [109] memoise_2.0.1      htmltools_0.5.9     lifecycle_1.0.5
#> [112] hardhat_1.4.2      httr_1.4.8          statnet.common_4.13.0

```

References

- Jay Barney. Firm resources and sustained competitive advantage. *Journal of Management*, 17(1):99–120, 1991.
- Oliver Baumann. **nkmodel**: Simulating the evolution of NK fitness landscapes, 2019. URL <https://CRAN.R-project.org/package=nkmodel>. R package.
- Oliver Baumann, Jan Schmidt, and Nils Stieglitz. Effective search in rugged performance landscapes: A review and outlook. *Journal of Management*, 45(1):285–318, 2019.
- Lawrence E. Blume. The statistical mechanics of strategic interaction. *Games and Economic Behavior*, 5(3):387–424, 1993.
- Stephen P. Borgatti and Martin G. Everett. Network analysis of 2-mode data. *Social Networks*, 19(3):243–269, 1997.
- Brantly Callaway and Pedro H. C. Sant’Anna. Difference-in-differences with multiple time periods. *Journal of Econometrics*, 225(2):200–230, 2021. doi: 10.1016/j.jeconom.2020.12.001.
- Stephen Downing. The architecture of rivalry and imitation mirages. *Under review at Management Science*, 2026.
- Sendil K. Ethiraj and Daniel A. Levinthal. Modularity and innovation in complex systems. *Management Science*, 50(2):159–173, 2004.
- Giovanni Gavetti and Daniel A. Levinthal. Looking forward and looking backward: Cognitive and experiential search. *Administrative Science Quarterly*, 45(1):113–137, 2000.
- Steve Hanneke, Wenjie Fu, and Eric P. Xing. Discrete temporal models of social networks. *Electronic Journal of Statistics*, 4:585–605, 2010.
- Stuart A. Kauffman and Simon Levin. Towards a general theory of adaptive walks on rugged landscapes. *Journal of Theoretical Biology*, 128(1):11–45, 1987.
- Stuart A. Kauffman and Edward D. Weinberger. The NK model of rugged fitness landscapes and its application to maturation of the immune response. *Journal of Theoretical Biology*, 141(2):211–245, 1989.
- Johan Koskinen and Christofer Edling. Modelling the evolution of a bipartite network: Peer referral in interlocking directorates. *Social Networks*, 34(3):309–322, 2012.
- Matthieu Latapy, Clémence Magnien, and Nathalie Del Vecchio. Basic notions for the analysis of large two-mode networks. *Social Networks*, 30(1):31–48, 2008.
- Michael J. Lenox, Scott F. Rockart, and Arie Y. Lewin. Interdependency, competition, and industry dynamics. *Management Science*, 52(5):751–766, 2006.
- Daniel A. Levinthal. Adaptation on rugged landscapes. *Management Science*, 43(7):934–950, 1997.
- Dean Lusher, Johan Koskinen, and Garry Robins. *Exponential Random Graph Models for Social Networks*. Cambridge University Press, 2013.
- Manim Community. Manim: Mathematical animation framework, 2022. URL <https://www.manim.community/>.
- Ruth M. Ripley, Tom A. B. Snijders, Zsófia Boda, András Vörös, and Paulina Preciado. *Manual for RSiena*. University of Oxford, Department of Statistics; Nuffield College, 2022. URL https://www.stats.ox.ac.uk/~snijders/siena/RSiena_Manual.pdf.
- Jan W. Rivkin. Imitation of complex strategies. *Management Science*, 46(6):824–844, 2000.

- Jan W. Rivkin and Nicolaj Siggelkow. Patterned interactions in complex systems: Implications for exploration. *Management Science*, 53(7):1068–1085, 2007. doi: 10.1287/mnsc.1060.0626.
- Nicolaj Siggelkow and Jan W. Rivkin. Speed and search: Designing organizations for turbulence and complexity. *Organization Science*, 16(2):101–122, 2005.
- Tom A. B. Snijders. Stochastic actor-oriented models for network change. *Journal of Mathematical Sociology*, 21(1–2):149–172, 1996.
- Tom A. B. Snijders, Gerhard G. van de Bunt, and Christian E. G. Steglich. Introduction to stochastic actor-based models for network dynamics. *Social Networks*, 32(1):44–60, 2010.
- David J. Teece, Gary Pisano, and Amy Shuen. Dynamic capabilities and strategic management. *Strategic Management Journal*, 18(7):509–533, 1997.
- Peng Wang and Philippa Pattison. Exponential random graph (p^*) models for affiliation networks. *Social Networks*, 31(1):12–25, 2009.
- Uri Wilensky. NetLogo. Center for Connected Learning and Computer-Based Modeling, Northwestern University, Evanston, IL, 1999. URL <http://ccl.northwestern.edu/netlogo/>.